1. What exactly is a service boundary?

A **service boundary** is the clear separation line that defines **what responsibilities, business rules, data, and decisions belong inside a particular service** in a microservice architecture.

Think of it as a border of ownership.

Inside the boundary:

- The service owns its business logic.
- The service owns its data.
- The service controls its rules and decisions.
- Other services cannot directly manipulate its internal state.

Outside the boundary:

- Other services can only interact through a defined interface, usually APIs or events.

Example:

An **Order Service** boundary might contain:

``` Order Service

Owns: - Creating orders - Updating order status - Order validation rules - Order history

Does not own: - Payment processing - Inventory calculation - Shipping management ```

The boundary answers one fundamental question:

> "What business capability does this service completely own?"

---

# 2. What business capability should belong inside one service?

A service should contain a **cohesive business capability**.

A business capability is something the business can recognize as a meaningful responsibility.

Examples:

Good boundaries:

```
Customer Service | ├── Customer registration ├── Customer profile ├── Customer preferences

Payment Service | ├── Payment processing ├── Refund handling ├── Payment status
```

Bad boundaries:

``` UserService

```
createUser()
sendEmail()
calculateInvoice()
checkInventory()
generateReport()
```

```

Why?

Because these are different business capabilities:

- Identity management
- Communication
- Billing
- Inventory

They change for different reasons.

A good service owns a capability that has:

- Its own business rules
- Its own language
- Its own lifecycle
- Its own data
```

3. How do we discover a good service boundary?

There is no mathematical formula. We discover boundaries by understanding the business.

Common techniques:

1. Domain analysis

Ask:

- What does the business do?
- What are the major activities?
- Who owns each activity?

Example:

Online shopping:

```
Customer Product Inventory Order Payment Delivery
```

These naturally become candidates.

---

2. Event Storming

Gather developers and domain experts.

Identify:

- Commands
- Events
- Aggregates
- Business rules

Example:

Command:

```
Place Order
```

Event:

```
Order Created
```

This reveals Order as a separate responsibility.

---

3. Look at business language

If people use different words, they may represent different contexts.

Example:

"Account"

Banking:

```
Account = money storage
```

CRM:

```
Account = customer organization
```

Same word, different meaning.

---

4. Analyze change patterns

Ask:

"Which things usually change together?"

If two things always change together, they probably belong together.

---

4. What signals indicate that two responsibilities belong to the same service?

They likely belong together when:

1. They share business rules

Example:

```
Order Creation + Order Validation
```

Both depend on order rules.

---

2. They change together

Example:

A shipping company changes:

```
Shipping price calculation Shipping rules Delivery options
```

at the same time.

Keep them together.

---

3. They share the same business language

Example:

Inventory:

```
Stock Reservation Warehouse Availability
```

Same domain vocabulary.

---

4. They share the same data ownership

Example:

Customer:

```
Name Address Preferences Account status
```

belongs together.

---

5. What signals indicate that they should be separated?

Separate responsibilities when:

1. They change independently

Example:

Payment rules change weekly.

Shipping rules change monthly.

They should not live together.

---

2. Different teams own them

Conway's Law:

> System structure tends to follow communication structure.

If two teams work independently, boundaries should support that.

---

3. Different business language exists

Example:

"Product"

Marketing:

`Product = campaign item`

Warehouse:

`Product = physical stock`

Different contexts.

---

4. Different scaling needs

Example:

Search service:

`Millions of requests`

Payment:

`Few but critical requests`

Separate scaling needs.

---

6. Is a service boundary the same thing as a bounded context?

No.

They are related but different.

Bounded Context

A concept from Domain Driven Design.

It defines:

> A boundary where a particular domain model has a specific meaning.

Example:

``` Sales Context

Customer means: Buyer

Support Context

Customer means: Person needing help ```

---

### Service Boundary

A technical boundary.

It defines:

`Deployment Ownership API Data Runtime`

Relationship:

`Bounded Context | ↓ Possible Microservice | ↓ Service Boundary`

Usually:

`1 Bounded Context ≈ 1 Service`

But not always.

---
```

7. Is one bounded context always one microservice?

No.

There are different possibilities.

Option 1:

One bounded context → One microservice

Common:

``` Payment Context

    |
    ↓
```

Payment Service ```

Option 2:

One bounded context → Multiple services

Example:

Large banking domain:

``` Banking Context

```
Account Service
Loan Service
Transaction Service
```

```

Option 3:

Multiple contexts temporarily inside one service

During migration:

``` Legacy Application

Customer Context Order Context Payment Context ```

All together.

---

# 8. What is the relationship between organizational boundaries and service boundaries?

Very important concept.

Conway's Law says:

> Organizations design systems that mirror their communication structure.

Example:

Company:

``` Team A Customer

Team B Payment

Team C Shipping ```

Architecture:

``` Customer Service

Payment Service

Shipping Service ```
```

Good alignment:

```
One team owns one service
```

Problems happen when:

```
5 teams | ↓ one giant service
```

Everyone depends on everyone.

---

9. How does data ownership help determine service boundaries?

Data ownership is one of the strongest signals.

Rule:

> A service should own the data it controls.

Example:

Order Service owns:

``` Orders table

Order_ID Customer_ID Status Items ```

Payment Service owns:

``` Payments table

Payment_ID Amount Transaction_Status ```

Bad design:

``` Order Service | | ↓ Shared Database

Orders Payments Customers Inventory ```

Now every service can modify everything.

Boundaries disappear.

---

# 10. Why should two microservices normally not share the same database?

Because database sharing creates hidden coupling.

Example:

```
Order Service | | ↓ Customer Database ↑ | Customer Service
```

Problems:

### 1. Cannot change independently

Customer Service changes table:

```
Rename customer_name
```

Order Service breaks.

---

2. No clear ownership

Who owns the data?

---

3. Tight deployment dependency

One database change affects many services.

```

Better:

``` Order Service

Order DB

Customer Service

Customer DB ```

Communication:

```
API Event Message Queue
```

---

# 11. What happens when a service boundary is too large?

A large boundary creates a **mini-monolith**.

Problems:

- Difficult deployment
- Large codebase
- More team conflicts
- Hard scaling
- Slow changes

Example:

``` Commerce Service

Customer Order Payment Inventory Shipping ```

This is basically a monolith wearing a microservice label.

---

# 12. What happens when it is too small?

You create too many tiny services.

Problems:

- Too much network communication
- Difficult debugging
- Operational complexity
- More failures

Example:

Bad:

``` Name Service

Address Service

Phone Service

Email Service ```

These are not real business capabilities.

---

# 13. What is a distributed monolith?

A distributed monolith is a system that looks like microservices but behaves like a monolith.

Characteristics:

```
Many services + High dependency + Must deploy together + Shared database + Many
synchronous calls
```

Example:

```
Order Service | ↓ Payment Service | ↓ Inventory Service
```
```

If one fails, everything fails.

---

14. How can bad service boundaries create a distributed monolith?

Example:

You split incorrectly:

``` Order Creation Service

Order Validation Service

Order Calculation Service ```

Now:

Creating an order requires:

```
Order Creation | ↓ Validation | ↓ Calculation | ↓ Database
```

Every request crosses many services.

The system becomes:

```
Distributed but tightly coupled
```

---

15. How do transactional requirements influence service boundaries?

Very strongly.

A transaction usually belongs inside one service.

Example:

Order creation:

```
Create Order Reserve Inventory Charge Payment
```

Should these be one transaction?

If yes:

Maybe boundary is too small.

Modern microservices avoid distributed transactions.

Instead:

Use:

- Events
- Saga Pattern
- Eventual consistency

Example:

``` Order Created

↓

Inventory Reserved

↓

Payment Completed ```

---

## 16. How do business changes reveal whether a boundary was designed correctly?

A good boundary absorbs change.

Example:

Payment rules change.

Good design:

```
Payment Service changes only
```

Bad design:

``` Order Customer Invoice Shipping

all modified ```

A good question:

> "When this business rule changes, how many services need modification?"

The answer tells you about your boundary quality.

---

# 17. How would I identify service boundaries in an Order Management system?

Start with business capabilities.

Possible decomposition:

``` Order Management System

Customer Service

Owns: - Customer profile - Customer information

Order Service

Owns: - Order creation - Order lifecycle - Order status

Inventory Service

Owns: - Stock - Availability - Reservation

Payment Service

Owns: - Payment - Refund - Transactions

Shipping Service

Owns: - Delivery - Tracking ```

Flow:

``` Customer places order

↓

Order Service (Create order)

↓

Inventory Service (Reserve stock)

↓

Payment Service (Process payment)

↓

Shipping Service (Deliver product) ```

Each service has:

- Its own data
- Its own rules
- Its own owner
- Its own language

That is the essence of a good service boundary.
```

Core principle to remember:

> A microservice boundary is not a technical decision first. It is a business ownership decision.

The best boundaries are where **business responsibility, data ownership, team ownership, and change patterns naturally meet**.

$$\text{Bounteded context}$$

---

2. Bounded Context

1. What is a bounded context?

A **bounded context** is a clearly defined boundary inside a business domain where a **specific domain model, language, and rules have a particular meaning**.

In simple words:

> A bounded context defines where a certain meaning of a business concept is valid.

Inside the boundary:

- Terms have a specific meaning.
- Business rules are consistent.
- Models represent the needs of that context.

Outside the boundary:

- The same term may have a different meaning.
- The model may be completely different.

Example:

In an e-commerce system:

```
Customer
```

does not necessarily represent the same thing everywhere.

Sales context:

```
Customer = person who buys products
```

Billing context:

```
Customer = person responsible for payment
```

Support context:

```
Customer = person requesting assistance
```

Same word, different models.

That boundary is a **bounded context**.

---

2. Why is a bounded context necessary in complex domains?

Because large businesses contain complexity.

Without boundaries, everyone tries to create one universal model.

Example:

A company creates:

```
Customer Object
```

with:

```
Name Address Phone Payment Details Order History Support Tickets Marketing Preferences
Shipping Address
```

Soon this object becomes huge.

Why?

Because different departments need different views of the customer.

Sales wants:

```
Customer | ├── Interested products ├── Buying history └── Sales opportunities
```

Billing wants:

```
Customer | ├── Invoice information ├── Tax details └── Payment status
```

Support wants:

```
Customer | ├── Complaints ├── Tickets └── Communication history
```

A bounded context prevents this confusion.

It allows:

- Independent models
- Clear ownership
- Better business alignment
- Easier changes

---

3. What exactly is bounded inside the context?

A bounded context contains:

1. Domain Model

The representation of business concepts.

Example:

``` Order Context

Order OrderItem OrderStatus PricingRule ```

---

## 2. Business Rules

Rules that apply only inside that context.

Example:

Sales:

```
Discount can be applied for premium customers
```

Billing:

```
Invoice must follow tax regulations
```

Different rules.

---

3. Language

The meaning of words.

Example:

"Account"

Banking Context:

```
Account = bank account holding money
```

CRM Context:
```

```
Account = business customer organization
```

---

4. Data Ownership

The context owns the data required for its responsibility.

Example:

Customer Support owns:

```
Customer Ticket Issue History Support Status
```

It does not own:

```
Payment History
```

---

4. How does meaning change between bounded contexts?

The meaning of a concept depends on the business purpose.

Example:

The word:

```
Product
```

Sales Context:

Product means:

```
Something available for selling
```

Contains:

```
Name Description Price Promotion
```

Inventory Context:

Product means:

```
A physical item stored in warehouse
```

Contains:

```
SKU Quantity Location Stock Level
```

Marketing Context:

Product means:

```
Something to promote
```

Contains:

```
Campaign Audience Brand Message
```

Same word.

Different reality.

---

5. Why can the same term have different meanings in different contexts?

Because different parts of an organization have different goals.

Example:

A hospital:

`Patient`

Doctor:

`Patient = person receiving medical treatment`

Billing department:

`Patient = person responsible for payment`

Research department:

`Patient = anonymous medical data source`

The business purpose changes the meaning.

This is called:

Ubiquitous Language

The language shared by people inside one context.

---

6. How is bounded context different from microservice?

They are related but not identical.

Bounded Context

A domain concept.

It answers:

> "Where does this model and language make sense?"

Microservice

A technical implementation.

It answers:

> "What independently deployable software component runs this capability?"

Relationship:

`Bounded Context | ↓ Possible Microservice`

Example:

``` Payment Bounded Context

↓

Payment Microservice ```

But:

`Bounded Context ≠ Always Microservice`

---

A company may have:

``` One Bounded Context

↓

Two Microservices ```

Example:

Large payment domain:

``` Payment Context
```

```
Payment Processing Service

Fraud Detection Service
```

---

7. How is bounded context different from subdomain?

This is a very important DDD concept.

Subdomain

A part of the business problem space.

It answers:

> "What area of the business exists?"

Example:

Online shopping business:

``` Business Domain

| ├── Sales ├── Inventory ├── Payment ├── Shipping ```

These are subdomains.

---

## Bounded Context

A solution boundary.

It answers:

> "How do we model this area in software?"

Relationship:

``` Business Domain

↓

Subdomain

↓

Bounded Context

↓

Software Implementation ```

Example:

Subdomain:

```
Customer Management
```

Bounded Context:

```
CRM Context Customer Support Context
```

---

8. Can one bounded context contain multiple aggregates?

Yes.

A bounded context usually contains multiple aggregates.

Example:

Order Management Context:

``` Bounded Context: Order
```

Aggregates:

```
Order Aggregate
Shipment Aggregate
Return Aggregate
```

Each aggregate protects its own business rules.

Relationship:

``` Bounded Context

```
  |
  |
  ├── Aggregate 1
  |
  ├── Aggregate 2
  |
  └── Aggregate 3
```

---

# 9. Can a microservice contain multiple bounded contexts?

Yes, technically possible.

Example:

During migration from a monolith:

``` Commerce Service

```
Customer Context
Order Context
Payment Context
```

Everything is inside one deployment.

However, this can become difficult because:

- Contexts may become coupled.
- Teams may conflict.
- Models may mix.

Usually:

`One bounded context per microservice`

is a good starting point.

---

10. Can multiple microservices implement one bounded context?

Yes.

Large bounded contexts may need multiple services.

Example:

Banking Context:

``` Banking Bounded Context

```
    |
    |
    ├── Account Service
    |
    ├── Transaction Service
    |
    └── Statement Service
```

All services belong to the same business model.
```

11. How do teams discover bounded contexts?

Teams discover them by understanding the business.

Common techniques:

---

1. Talk with domain experts

Ask:

- What are the major business activities?
- Who owns decisions?
- What terminology is used?

---

2. Event Storming

Identify:

Commands:

`Place Order Cancel Order Approve Payment`

Events:

`Order Created Payment Completed Order Cancelled`

Patterns reveal boundaries.

---

3. Analyze business departments

Example:

Organization:

``` Sales Team

Finance Team

Warehouse Team

Support Team ```

Possible contexts:

``` Sales Context

Billing Context

Inventory Context

Support Context ```

---

## 4. Look for language differences

Different words often reveal different contexts.

---

# 12. What role does business language play?

Business language is one of the strongest signals in DDD.

Inside a bounded context:

Everyone should use the same language.

This is called:

**Ubiquitous Language**

Example:

Order Team:

```
Order Customer Shipment Cancellation
```

Everyone understands these words the same way.

But:

Customer Team:

```
Customer Profile Preference Communication
```

Different language.

Different model.

---

13. How does data ownership work across bounded contexts?

Each bounded context owns its own data.

Example:

``` Sales Context

Owns: Customer Interest Lead Status Purchase History ```

``` Billing Context

Owns: Invoice Payment Tax Information ```

``` Shipping Context

Owns: Address Delivery Status Tracking ```

No context directly changes another context's data.

Communication happens through:

- APIs
- Events
- Messages

---

# 14. How should bounded contexts communicate?

Through explicit contracts.

Common approaches:

## 1. API Communication

Example:

``` Order Context

    |
    |
    ↓

Inventory API ```

Request:

```
Reserve Product
```

Response:

```
Stock Reserved
```

---

2. Domain Events

Example:

Order Service publishes:

`OrderCreated Event`

Other contexts react:

``` Inventory Context

Payment Context

Shipping Context ```

---

### 3. Anti-Corruption Layer

When one context communicates with another, a translation layer can protect the internal model.

Example:

``` Legacy System

    |
    ↓

Anti-Corruption Layer

    |
    ↓

New System ```

---

## 15. What problems occur when boundaries are unclear?

Unclear boundaries create complexity.

### 1. Shared models everywhere

Example:

`Common Customer Object`

used by:

`Sales Billing Support Shipping`

Every change breaks everyone.

---

### 2. Database sharing

Example:

``` One Customer Table

Used by all teams ```

No ownership.

---

### 3. Tight coupling

Changing one business rule affects many areas.

---

### 4. Distributed monolith

Many services but no independence.

Example:

`Order Service | Payment Service | Customer Service`

Every change requires all services.

---
```

Good Example: Is Customer really the same object everywhere?

Let's analyze.

1. Sales Context

Goal:

Sell products.

Customer:

``` Customer

- Name
- Interests
- Purchase intention
- Leads
- Sales history ```

---

## 2. Billing Context

Goal:

Receive money.

Customer:

``` Customer

- Billing address
- Tax information
- Payment status
- Invoice history ```

---

## 3. Shipping Context

Goal:

Deliver products.

Customer:

``` Customer

- Delivery address
- Contact number
- Delivery preference ```

---

## 4. Support Context

Goal:

Solve problems.

Customer:

``` Customer

- Complaints
- Tickets
- Communication history
- Satisfaction score ```

---

Are these the same Customer?

Technically:

No.

They are different models representing the same real-world person from different perspectives.

In DDD:

``` Real World Customer

    |
    |
    ↓

Sales Customer

Billing Customer

Shipping Customer

Support Customer ```

---

# Core principle to remember:

> A bounded context is not a technical boundary. It is a boundary of meaning.

A good bounded context protects a model where:

- Words have clear meaning.
- Business rules are consistent.
- Data has clear ownership.
- Teams can work independently.

The goal is not to create many boundaries.

The goal is to create **the right boundaries where complexity naturally exists.**

---

$$\text{Ubiquitous language}$$

---

# 3. Ubiquitous Language

## 1. What is ubiquitous language?

**Ubiquitous Language** is a shared language created and used by **developers, domain experts, product owners, and business stakeholders** within a specific bounded context.

It means:

> Everyone involved in building the system uses the same business terms with the same meaning.

The goal is to remove translation gaps between business people and developers.

Example:

An e-commerce company uses the term:

```
id="u1" Order
```

Everyone should understand what Order means.

Business expert:

```
A confirmed request from a customer to purchase products.
```

Developer:

```
java Order order = orderService.createOrder();
```

Both refer to the same concept.

Without ubiquitous language:

Business says:

> "When a customer confirms a purchase, create a booking."
```

Developer thinks:

> "Booking means a reservation object."

Later confusion appears.

The software model and business model become different.

---

2. Why is language considered part of software architecture in DDD?

Because language shapes the design of the system.

In traditional development, we often think architecture means:

- Database design
- Framework selection
- Deployment
- Infrastructure

DDD adds another important layer:

> The structure of the software should reflect the structure of the business language.

Example:

Business language:

```
Customer places Order Order reserves Inventory Payment completes Transaction
```

Software design:

``` Customer Aggregate

Order Aggregate

Inventory Service

Payment Service ```

The words used by the business influence:

- Class names
- Service names
- API names
- Domain models
- Database concepts

Bad language creates bad architecture.

---

Example:

A developer creates:

```
java UserManager
```

But the business talks about:

```
Customer Subscriber Account Holder Buyer
```

Which one is correct?

It depends on the domain.

The code should represent the business meaning, not developer assumptions.

---

3. Who defines the ubiquitous language?

Nobody defines it alone.

It is created collaboratively by:

- Domain experts
- Product owners
```

- Developers
- Architects
- Business analysts

The domain experts provide:

```
Business knowledge
```

Developers provide:

```
Technical understanding
```

Together they create the shared language.

Example:

Banking team discussion:

Business expert:

> "A customer can open an account."

Developer asks:

> "What does open mean? Does it mean application submitted or account activated?"

The discussion creates precise language:

```
Account Requested Account Approved Account Activated
```

Now the model becomes clearer.

---

4. Should developers use business terminology directly in code?

Yes.

Developers should use domain terminology whenever possible.

Example:

Business term:

```
Order
```

Code:

```java class Order {

} ```

Business term:

```
Payment Authorization
```

Code:

```
java authorizePayment()
```

This makes the code understandable.

Bad example:

Business:

```
Customer places order
```

Code:

```
java processTransactionObject()
```

The code hides business meaning.

However, technical terms are still needed.
```

Example:

Good:

```
java OrderRepository
```

Because repository is a technical concept.

Bad:

```
java OrderDataManagerThing
```

because it mixes unclear technical naming.

---

5. Should class names reflect domain terminology?

Yes.

Class names should represent concepts from the domain.

Example:

Banking system:

Business language:

```
Account Transaction Loan Customer
```

Code:

```java class Account {}

class Transaction {}

class Loan {}

class Customer {} ```

This creates a direct connection between:

``` Business Model

    ↓

Software Model ```
```

Avoid generic names:

```
java Manager Handler Processor Helper Utility
```

when a meaningful domain name exists.

Example:

Bad:

```
java CustomerManager
```

Better:

```
java CustomerRegistrationService
```

if the responsibility is registration.

---

6. Should API terminology reflect domain terminology?

Yes.

APIs are communication contracts.

They should speak the language of the domain.

Example:
```

Good API:

`POST /orders/{id}/cancel`

because business says:

`Cancel Order`

Bad API:

`POST /orders/{id}/updateStatus`

Why?

Because "update status" is technical.

The business action is:

`Cancel Order`

Another example:

Payment domain:

Good:

`POST /payments/{id}/refund`

Bad:

`POST /payments/{id}/changeState`

The API should express business intent.

---

7. How is ubiquitous language related to bounded context?

They are tightly connected.

A bounded context defines:

> Where a particular language has a specific meaning.

The same term can exist in different bounded contexts.

Example:

`Customer`

Sales Context:

`Customer = buyer interested in products`

Support Context:

`Customer = person requesting help`

Billing Context:

`Customer = person responsible for payment`

Each context has its own ubiquitous language.

Relationship:

``` Bounded Context

↓

Defines the boundary

↓

Ubiquitous Language

↓
```

Defines the meaning inside that boundary ```

---

8. Can one term mean different things in different bounded contexts?

Yes.

This is normal in DDD.

Example:

Customer

Sales Context

``` Customer

- Leads
- Buying behavior
- Interests ```

---

### Billing Context

``` Customer

- Invoice information
- Payment responsibility
- Tax details ```

---

### Shipping Context

``` Customer

- Delivery address
- Contact information ```

---

They are not contradictions.

They are different models for different purposes.

---

The mistake is trying to create:

```
Universal Customer Object
```

for everything.

That usually creates a huge complicated model.

---

# 9. What happens when technical terminology leaks into domain discussions?

The business model becomes harder to understand.

Example:

Developer says:

> "We need to update the ORM entity state before triggering the event handler."

Business person thinks:

> "What does that mean?"

The conversation is now about technology instead of business.

---

Better:

Developer says:
```

"When the order is approved, we notify the warehouse."

Now everyone understands.

Technical language is useful inside technical discussions.

But domain discussions should use domain language.

---

10. How do ambiguous terms reveal hidden domain problems?

Ambiguous words often show that the business concept is not clearly understood.

Example:

The word:

```
Status
```

What does it mean?

Order status?

```
Created Paid Shipped Delivered
```

Payment status?

```
Pending Failed Completed
```

Customer status?

```
Active Inactive Blocked
```

One word hides multiple concepts.

The solution:

Split the language.

Instead of:

```
Status
```

use:

``` OrderStatus

PaymentStatus

CustomerStatus ```

---

Another example:

"Account"

Could mean:

```
Bank account User account Customer account Company account
```

The ambiguity reveals missing boundaries.

---

## 11. How should the language evolve when business rules change?

Ubiquitous language is not fixed forever.

It evolves with the business.

Example:

Old business concept:

```
Customer
```
```

New requirement:

The company introduces subscriptions.

Now:

`Customer Subscriber Member Account Holder`

may become separate concepts.

The model should change.

DDD principle:

> When the business language changes, the software model should evolve with it.

Example:

Old:

`java Customer.isPremium()`

New:

`java Subscription.isActive()`

The language changed, so the model changed.

---

12. How can developers, product owners, and domain experts keep the language synchronized?

Through continuous collaboration.

1. Domain discussions

Regular meetings:

`Developers + Product Owners + Domain Experts`

Discuss business concepts.

---

2. Maintain a domain glossary

Example:

``` Order: A confirmed request to purchase products.

Reservation: Temporary holding of inventory.

Payment: The process of transferring money. ```

Everyone follows the same meaning.

---

## 3. Review code terminology

Ask:

"Does this class name match the business language?"

Example:

Business says:

`Refund`

Code says:

`ReverseTransactionProcessor`

Mismatch.
```

Rename.

---

4. Use language in APIs and documentation

The same terms should appear in:

- Code
- API contracts
- User stories
- Documentation
- Tests

---

5. Encourage developers to question unclear words

A good developer does not silently accept vague requirements.

Example:

Business says:

> "Deactivate customer."

Developer asks:

> "Does deactivate mean prevent login, stop billing, or archive data?"

That question improves the domain model.

Core principle to remember:

> Ubiquitous Language is the bridge between business understanding and software design.

A strong DDD system has:

``` Business Words

↓

Domain Model

↓

Code

↓

APIs ```

All speaking the same language.

When the language is clear, the architecture becomes clearer. When the language is confused, the software usually becomes confused too.

---

$$\text{Domain Model}$$

---

# 4. Domain Model

### 1. What is a domain model?

A **domain model** is a software representation of the **business concepts, rules, behaviors, and relationships** that exist inside a specific business domain.

In simple words:

> A domain model is the way we represent how the business works inside our software.

It is not just data.
```

It includes:

- Business concepts
- Business rules
- Business behaviors
- Relationships
- Constraints

Example:

In an e-commerce system:

Business concepts:

```
id="0t5h6p" Customer Order Product Payment Shipment
```

A simple database approach may think:

```text Order Table
```

---

id customer_id status price ```

But a domain model thinks:

```id="q2x5pp" Order

Responsibilities: - Add items - Calculate total - Cancel order - Confirm order - Validate rules ```
```

The focus is:

> What can this business object do?

Not only:

> What data does it store?

---

2. What does a domain model represent?

A domain model represents the **business reality from the perspective of a particular bounded context**.

It represents:

1. Business Objects

Example:

```
id="7j8k4m" Customer Order Invoice Product
```

---

2. Business Relationships

Example:

```id="x2v4bk" Customer

places

    ↓

Order

contains

    ↓

Order Items ```
```

---

3. Business Rules

Example:

Business rule:

> An order cannot be cancelled after shipping.

Domain model:

```java public void cancel(){

if(status == SHIPPED){
    throw new OrderCannotBeCancelledException();
}

status = CANCELLED;

} ```

The model protects the rule.

---

### 4. Business Behavior

Example:

Instead of:

`java order.setStatus("CANCELLED");`

The domain model says:

`java order.cancel();`

The second one expresses business meaning.

---

## 3. Is a domain model just database entities?

No.

This is one of the biggest misunderstandings in application development.

A database entity represents:

How data is stored.

A domain model represents:

How the business works.

They can overlap, but they are not the same.

Example:

Database table:

``` orders

id customer_id amount status ```

Domain model:

```java class Order {

private List<OrderItem> items;

public Money calculateTotal(){

}

public void cancel(){

}

public void confirm(){

}

} ```

The domain model contains behavior.

The database does not.

---
```

4. What is the difference between a domain model and a persistence model?

Persistence Model

Concern:

How data is stored and retrieved.

Example:

JPA Entity:

```java @Entity class OrderEntity {

Long id;

String status;

BigDecimal amount;

} ```

It cares about:

- Database columns
- Table mapping
- Relationships
- ORM rules

---

## Domain Model

Concern:

How the business behaves.

Example:

```java class Order {

OrderStatus status;

public void cancel(){

    if(status == SHIPPED){
        throw new Exception();
    }

    status = CANCELLED;
}

} ```

It cares about:

- Business rules
- Decisions
- Behavior

---

Relationship:

``` Database

↓

Persistence Model

↓

Mapping

↓

Domain Model ```

They may look similar, but their responsibilities are different.

---
```

5. What is the difference between an anemic and rich domain model?

This is a very important DDD concept.

Anemic Domain Model

An anemic model contains mostly data with little or no behavior.

Example:

```java class Order {

private String status;

private double amount;

public void setStatus(String status){

    this.status = status;

}

} ```

Business logic lives somewhere else:

```java OrderService {

if(order.status.equals("NEW")){

    order.status="CANCELLED";

}

} ```

Problems:

- Business rules are scattered.
- Objects are just data containers.
- Harder to maintain.

---

# Rich Domain Model

A rich model contains data + behavior.

Example:

```java class Order {

private OrderStatus status;

public void cancel(){

    if(status == SHIPPED){

        throw new OrderException();

    }

    status = CANCELLED;

}

} ```

The object protects itself.

Benefits:

- Clear business logic
- Better maintainability
- Fewer invalid states

---

Comparison:

| Anemic Model | Rich Model | | ---------- | ---------- | | Data only | Data + behavior | | Logic in services | Logic inside domain objects | | Weak protection | Strong protection | | Procedural style | Object-oriented style |

---
```

6. Where should business rules live?

Business rules should live as close as possible to the domain model.

Example:

Rule:

A customer cannot place an order if their account is blocked.

Bad:

```java OrderService {

checkCustomer();

createOrder();

} ```

Better:

```java Customer {

public void verifyCanPlaceOrder(){

}

} ```

---

The exact location depends on the rule.

## Entity Rule

Example:

Order cannot be cancelled after shipping.

```
java Order.cancel()
```

Value Object Rule

Example:

Money cannot be negative.

```java Money {

constructor(){

if(amount < 0)

}

} ```

---

## Domain Service Rule

When a rule involves multiple objects:

Example:

```
java PricingService.calculateDiscount()
```

---

# 7. What does behavior-rich modeling mean?

Behavior-rich modeling means domain objects contain meaningful business actions.

Instead of:

```
java customer.setStatus("ACTIVE");
```
```

Use:

```
java customer.activate();
```

Instead of:

```
java order.setPaymentStatus("PAID");
```

Use:

```
java order.markAsPaid();
```

The difference:

First approach:

The caller controls the rules.

Second approach:

The object controls the rules.

Example:

Poor design:

```
java order.status = "SHIPPED";
```

Anyone can do it.

Rich design:

```
java order.ship();
```

Inside:

```java public void ship(){

if(paymentStatus != PAID){

throw new Exception();

}

status = SHIPPED;

} ```

The model protects itself.

---

# 8. Why should domain models protect business rules?

Because business rules are the heart of the application.

If rules are outside the model:

Example:

```java OrderController

OrderService

PaymentService

RandomHelper ```

Rules become scattered.

Different parts of the system may apply different rules.

Example:

One developer writes:

```
java if(order.status == NEW)
```

Another writes:

```
java if(order.status != SHIPPED)
```

Now inconsistent behavior exists.

A protected domain model ensures:

``` All paths

↓

Same business rules

↓

Valid business state ```

---

# 9. Should domain objects depend on Spring?

Normally, no.

Domain objects should be plain Java objects.

Example:

Good:

```java public class Order {

public void cancel(){

}

} ```

Avoid:

```java @Component public class Order {

} ```

Why?

Because Spring is infrastructure.

The domain should not know:

- Dependency injection
- Framework lifecycle
- Configuration

Benefits:

- Easier testing
- Less coupling
- Cleaner design

---

# 10. Should domain objects depend on repositories?

Normally, no.

Example:

Bad:

```java class Order {

@Autowired OrderRepository repository;

public void cancel(){

repository.save(this);

}

} ```

Why bad?

The domain now knows about persistence.

Better:

```java OrderService {

order.cancel();

repository.save(order);

} ```

The application layer coordinates.

The domain decides.

---

Responsibilities:

Domain:

`What should happen?`

Application:

`When should it happen?`

Infrastructure:

`How is it stored?`

---

# 11. Should domain objects know about HTTP, Kafka, JSON, JPA, etc.?

No.

These are infrastructure concerns.

Domain should not know:

`HTTP REST Kafka JSON Database JPA`

Example:

Bad:

```java class Order {

@JsonProperty @Column @Entity

} ```

The object is now tied to technology.

---

Better:

``` Controller

↓

Application Layer

↓

Domain Model

↓

Infrastructure ```

The domain stays pure.
```

12. How do entities, value objects, aggregates, services, events, and invariants combine to form the model?

Together they create the complete business model.

Example: Order Domain

Entity

Object with identity.

Example:

``` Order

Order ID = 12345 ```

Even if values change, it remains the same order.

---

## Value Object

Object defined by its value.

Example:

``` Money

$100 USD ```

No identity.

---

## Aggregate

A group of objects treated as one consistency boundary.

Example:

``` Order Aggregate

Order

| ├── OrderItem

└── ShippingAddress ```

---

## Aggregate Root

The entry point.

Example:

``` Order

controls:

OrderItem ```

Outside objects cannot directly modify OrderItem.

---

## Domain Service

Business logic that does not naturally belong to one object.

Example:

``` CurrencyConversionService
```

DiscountCalculationService ```

Domain Event

Something important happened.

Example:

``` OrderPlaced

PaymentCompleted

ShipmentCreated ```

---

### Invariant

A rule that must always remain true.

Example:

``` Order total cannot be negative.

Paid order cannot become unpaid. ```

---

Together:

``` Domain Model

```
|
├── Entities
├── Value Objects
├── Aggregates
├── Domain Services
├── Domain Events
└── Invariants
```

```

---

13. What makes a domain model maintainable?

A maintainable domain model has:

1. Clear responsibilities

Each object has one meaningful responsibility.

---

2. Good naming

Uses ubiquitous language.

Example:

Good:

```
java approveLoan()
```

Bad:

```
java updateFlag()
```

---

3. Protected business rules

Invalid states are difficult to create.

---

4. Low coupling

Domain does not depend on infrastructure.

---

5. High cohesion

Related behaviors stay together.

Example:

Order handles:

```
Cancel Confirm Calculate Total
```

Not:

```
Generate PDF Send Email Save Database
```

---

14. When is sophisticated domain modeling unnecessary?

DDD is powerful, but not every application needs a complex model.

Avoid heavy modeling when the system is simple.

Examples:

Simple CRUD Application

Example:

Employee directory:

```
Create Employee Update Employee Delete Employee Search Employee
```

A simple CRUD model may be enough.

Reporting System

Example:

Dashboard:

```
Read data Generate charts Export reports
```

Complex domain modeling adds unnecessary complexity.

Simple Configuration Service

Example:

```
Store application settings
```

No complex business rules.

Use sophisticated domain modeling when:

- Business rules are complex.
- Domain knowledge is important.
- Many decisions exist.
- The system will evolve for years.

Core principle to remember:

> A domain model is not a representation of your database. It is a representation of your business.

A strong domain model answers:

``` What does the business do?
```

Why is this rule needed?

Who owns this decision?

How do we prevent invalid states? ```

The goal is not to create many classes.

The goal is to create a model where the code speaks the language of the business.

$$\text{Entity}$$

---

5. Entity

1. What is a domain entity?

A **domain entity** is an object in the domain model that is defined primarily by its **unique identity**, not only by its attributes.

In simple words:

> An entity is something that remains the same thing even when its internal data changes.

Example:

An order:

```
id="e1" Order #12345
```

Today:

```
text Status = Pending Amount = $100
```

Tomorrow:

```
text Status = Paid Amount = $120
```

The data changed, but it is still the same order.

Why?

Because its identity remains:

```
text Order ID = 12345
```

Common domain entities:

```
text Customer Order Account Subscription Employee Bank Transaction
```

They have a lifecycle.

They are created, modified, and sometimes deleted.

---

2. What makes something an entity rather than a value object?

The key difference is **identity**.

An entity has:

```
id="a7" Identity + Attributes + Behavior
```

A value object has:

```
id="b8" Value + Meaning
```

Example:

Entity

```text Customer

ID: CUST-101

Name: John

Email: john@test.com ```

Two customers can have the same name:

```text Customer A: John

Customer B: John ```

They are still different people.

Identity matters.

---

### Value Object

```text Money

Amount: 100

Currency: USD ```

Another:

```text Money

Amount: 100

Currency: USD ```

They are equal.

There is no need to know:

```
text Money Object #1 Money Object #2
```

The value defines the identity.

Rule:

> If changing attributes still keeps it the same thing, it is probably an entity.

---

3. Why is identity important?

Identity allows the system to track something through its lifecycle.

Example:

Order:

Day 1:

```
text Order ID: 5001 Status: CREATED
```

Day 5:

```
text Order ID: 5001 Status: SHIPPED
```

The system knows:

"This is the same order."

Without identity:

How would we know?

```
text Order: John Laptop $1000
```

After update:

```
text Order: John Laptop $1200
```

Is it the same order?

Identity solves this.

Identity is important for:

- Tracking changes
- Maintaining relationships
- Business decisions
- Audit history
- Lifecycle management

---

4. Does an entity's identity depend on its database primary key?

No.

This is a common misunderstanding.

A database primary key is a **technical identifier**.

A domain identity is a **business concept**.

Sometimes they are the same.

Example:

```
java OrderId = 10001
```

Database:

```
sql PRIMARY KEY = 10001
```

Fine.

But they do not have to be.

Example:

Bank account:

Business identity:

```
text Account Number: 123456789
```

Database:

```
text Internal ID: 987654
```

The database ID is only an implementation detail.

DDD question:

Ask:

> Does the business care about this identity?

If yes, it is a domain identity.

---

5. Can an entity change over time while maintaining the same identity?

Yes.

Actually, this is one of the main characteristics of an entity.

Example:

Customer:

Created:

```text Customer ID: C100

Name: Rahim

Status: Active ```

After years:

```text Customer ID: C100

Name: Rahim Ahmed

Status: Premium ```

The attributes changed.

Identity stayed the same.

---

Entities have:

```text Stable identity

+ Changing state ```

---

# 6. What constitutes entity equality?

Entity equality is based on identity.

Example:

```java Customer customer1

ID = 101

Customer customer2

ID = 101 ```

Even if:

```
text Name = Different Address = Different
```

They represent the same entity.

Equality:

```
text customer1 == customer2
```

because:

```
text Identity is same
```

Value object equality is different.

Money:

```text $100 USD

= $100 USD ```

because values are the same.

---

Comparison:

| Entity | Value Object | | ---------- | ---------- | | Identity based | Value based | | Changes over time | Usually immutable | | Has lifecycle | No lifecycle | | Example: Order | Example: Money |

---
```

7. Should entities expose setters?

Generally, no.

Public setters create uncontrolled state changes.

Bad:

```
java order.setStatus("SHIPPED");
```

Anyone can change the order.

Problem:

What if:

```
text Payment not completed?
```

The order should not become shipped.

Better:

```
java order.ship();
```

Inside:

```java public void ship(){
if(paymentStatus != PAID){
    throw new OrderException();
}
status = SHIPPED;
} ```
```

The entity controls its state.

Instead of:

```
text Change data directly
```

Prefer:

```
text Perform business action
```

---

8. How should entity state transitions be controlled?

Through behavior methods.

An entity should expose meaningful actions.

Example:

Order lifecycle:

```text CREATED
↓
CONFIRMED
↓
PAID
↓
SHIPPED
↓
DELIVERED ```
```

Bad:

```
java order.status = DELIVERED;
```

Good:

```
java order.markDelivered();
```

Inside:

```java public void markDelivered(){
if(status != SHIPPED){
throw new InvalidStateException();
}
status = DELIVERED;
} ```

The entity protects valid transitions.

---

# 9. Where should entity-specific business rules live?

Inside the entity itself.

Example:

Rule:

> A bank account cannot withdraw more money than its balance.

Bad:

```
java AccountService.withdraw(account, amount);
```

Better:

```
java account.withdraw(amount);
```

Inside:

```java public void withdraw(Money amount){
if(balance.isLessThan(amount)){
throw new InsufficientBalanceException();
}
balance = balance.subtract(amount);
} ```

The entity owns the rule.

---

Examples:

Order:

```
text cancel() confirm() ship()
```

Customer:

```
text activate() block() upgrade()
```

Subscription:

```
text renew() pause() cancel()
```

---

10. What should an entity constructor guarantee?

An entity constructor should create a valid object.

It should prevent invalid states.

Bad:

```
java Order order = new Order();
```

Now:

```
text Order has no ID No status No customer
```

Invalid.

Better:

```
java Order order = new Order(customerId, items);
```

Constructor guarantees:

```text Order has:

✓ Identity ✓ Required information ✓ Valid initial state ```

---

Example:

Customer:

```
java new Customer(email);
```

Should validate:

```
text Email exists Email format valid Status initialized
```

---

11. Can entities exist outside aggregates?

Technically yes.

But in DDD, entities usually belong inside an aggregate.

Example:

Order aggregate:

```text Order (Entity)

| | ├── OrderItem (Entity) | └── ShippingAddress (Value Object) ```

The Order is the aggregate root.

Outside objects access:

```
java order.addItem()
```

not:

```
java orderItem.changePrice()
```

An entity without aggregate protection can create consistency problems.

---

12. What is the difference between a DDD entity and a JPA `@Entity`?

Very important.

They are different concepts.

JPA Entity

Technical concept.

Example:

```java @Entity class OrderEntity {

Long id;

String status;

} ```

Purpose:

```
text Database mapping
```

---

### DDD Entity

Business concept.

Example:

```java class Order {

OrderId id;

public void cancel(){

}

public void ship(){

}

} ```

Purpose:

```
text Business behavior
```

A JPA entity may not be a domain entity.

Example:

Database table:

```
text USER_LOGIN_HISTORY
```

JPA entity:

```
java LoginHistoryEntity
```

But business may not consider it an entity.

---

13. Should every database table become a domain entity?

No.

This is a common mistake.

Database design:

```
text Tables
```

Domain design:

```
text Business concepts
```

They are different.

Example database:

```text ORDER_TABLE

ORDER_ITEM_TABLE

ORDER_HISTORY_TABLE

ORDER_STATUS_TABLE ```

Domain model:

```text Order Aggregate

Order

OrderItem

```

The database has more tables than domain entities.

Rule:

> Do not design the domain model from the database schema.

Design from business behavior.

---

14. How should entity identity be generated?

There are several approaches.

---

1. Database generated ID

Example:

```text Auto increment:

10001 10002 10003 ```

Advantages:

- Simple
- Fast
- Database handles generation

Disadvantages:

- Database dependency
- Difficult in distributed systems

---

## 2. UUID

Example:

```
text 550e8400-e29b-41d4-a716
```

Advantages:

- Globally unique
- Good for microservices
- Can generate before saving

Disadvantages:

- Larger storage size
- Less readable

---

3. Domain-specific ID

Example:

```text Order:

ORD-2026-000123 ```

Advantages:

- Business friendly
- Meaningful

Disadvantages:

- Requires design

---

The choice depends on the domain.

---

# 15. UUID vs sequence vs domain-specific ID: what are the trade-offs?

## UUID

Example:

```text 8f4c-92ab-33de
```

Pros:

✓ Distributed friendly ✓ No central generator needed ✓ Good for microservices

Cons:

✗ Harder to read ✗ Larger indexes

Database Sequence

Example:

```text 10001 10002 10003
```

Pros:

✓ Simple ✓ Fast ✓ Small storage

Cons:

✗ Database dependent ✗ Harder across multiple systems

Domain-specific ID

Example:

```text CUSTOMER-BD-000123
```

Pros:

✓ Human readable ✓ Business meaningful

Cons:

✗ More design effort ✗ Generation rules required

Microservice systems often prefer:

```text UUID internally

+ Domain ID externally ```

Example:

Internal:

```text UUID: a83b-92ff
```

Business:

```
text Order Number: ORD-2026-00123
```

Example Analysis

Why is Order an entity but Money probably isn't?

Order:

```
text Order ID: ORD-1001
```

Changes:

```text Status: Created → Paid → Shipped

Amount: $100 → $120 ```

Still the same order.

Therefore:

```
text Order = Entity
```

---

## Money:

```text Amount: 100

Currency: USD ```

If another object has:

```text Amount: 100

Currency: USD ```

They are identical.

No one asks:

"Which $100 object is this?"

Therefore:

```
text Money = Value Object
```

Core principle to remember:

> An entity is defined by who it is, not what it contains.

A strong entity:

```
text Has identity + Controls its behavior + Protects business rules + Maintains valid state + Represents a real business concept
```

The question is not:

> "Does this have a database table?"

The real question is:

> "Does the business care about its identity over time?"

$$\text{Value object}$$

```

6. Value Object

1. What is a value object?

A **Value Object** is an object in the domain model that is defined by its **value rather than its identity**.

In simple words:

> A value object represents a concept where only the information it contains matters, not which specific object it is.

Examples:

```
text Money EmailAddress Address Percentage DateRange Quantity
```

A value object usually:

- Has no unique identity.
- Is immutable.
- Represents a meaningful concept.
- Contains validation rules.
- Contains behavior related to its value.

Example:

Instead of:

```
java BigDecimal price; String currency;
```

A domain model uses:

```
java Money price;
```

Because money is not just a number.

Money has:

```
text Amount Currency Rules Behavior
```

Example:

```
java Money salary = new Money(5000, "USD");
```

The object represents a complete business concept.

---

2. Why does a value object not require identity?

Because the value itself defines what it is.

Example:

```
text Money: 100 USD
```

Another object:

```
text Money: 100 USD
```

Are they different?

No.

They represent the same value.

Nobody asks:

> "Is this the original $100 object or another $100 object?"

Identity is meaningless here.

Compare with Customer:

```text Customer A
```

ID: 1001

Name: Rahim ```

Another:

```text Customer B

ID: 1001

Name: Karim ```

Something is wrong.

Customers need identity.

---

Rule:

Entity:

```
text Who are you?
```

Value Object:

```
text What value do you represent?
```

---

3. How is equality determined?

Value objects are compared by their attributes.

Example:

```java Money money1 = new Money(100, "USD");

Money money2 = new Money(100, "USD"); ```

Equality:

```
text money1 == money2
```

because:

```text Amount = 100

Currency = USD ```

are the same.

---

For an Address:

```text Address 1:

Street: Main Road

City: Dhaka ```

```text Address 2:

Street: Main Road

City: Dhaka ```

They are equal because their values are equal.

---

Entity equality:

```
text Same identity
```

Value Object equality:

```
text Same values
```

```

4. Why should value objects normally be immutable?

Immutable means:

> Once created, the object's internal state cannot change.

Example:

Bad:

```java Money money = new Money(100, "USD");

money.setAmount(200); ```

Now the same object changed meaning.

This creates problems.

---

Better:

```java Money money = new Money(100, "USD");

Money newMoney = money.add(new Money(50,"USD")); ```

The original remains unchanged.

---

Benefits of immutability:

## 1. Safer code

A value cannot unexpectedly change.

---

## 2. Easier reasoning

Example:

```
text Money = 100 USD
```

Always means:

```
text 100 USD
```

---

3. Thread safety

Multiple threads can safely use immutable objects.

---

4. Fewer side effects

Changes create new values instead of modifying existing ones.

---

5. What business rules should a value object enforce?

A value object should protect rules related to its own meaning.

Example:

Money

Rules:

```text Amount cannot be negative.

Currency must exist.

Cannot add different currencies. ```

Example:
```

```
java Money money = new Money(-100,"USD");
```

Should fail.

EmailAddress

Rules:

```text Email cannot be empty.

Email format must be valid. ```

---

### Percentage

Rules:

```
text Percentage must be between 0 and 100.
```

Example:

```
java new Percentage(150);
```

Invalid.

The object protects its own correctness.

---

6. Why is Money better than using BigDecimal everywhere?

Because:

```
java BigDecimal amount;
```

only represents a number.

But money has business meaning.

Example:

Bad:

```
java BigDecimal price;
```

Questions:

- Which currency?
- Can it be negative?
- How should rounding work?
- Can USD and EUR be added?

Better:

```
java Money price;
```

Now:

```java Money {

amount;

currency;

add();

subtract();

convert();

} ```
```

The model understands money.

Example:
```

Without Money:

```
java total = price + tax;
```

Problem:

Maybe:

```text price = 100 USD

tax = 10 EUR ```

Invalid.

---

With Money:

```
java price.add(tax);
```

The object can prevent invalid operations.

---

# 7. Why is EmailAddress sometimes better than String?

Because String is too generic.

Example:

```
java String email;
```

A String can contain:

```text hello

12345

abc@ ```

Nothing prevents invalid data.

---

Better:

```
java EmailAddress email;
```

Now:

```java EmailAddress {

validateFormat();

normalize();

} ```

Example:

Input:

```
text USER@GMAIL.COM
```

Can become:

```
text user@gmail.com
```

The business rule lives in one place.

Instead of:

```
java if(email.contains("@"))
```

everywhere:

Use:

```
java email.isValid()
```
```

8. When should primitive values be replaced with domain-specific value objects?

When a primitive represents an important business concept.

This is called:

Primitive Obsession

Example:

Bad:

```java String customerId;

String email;

BigDecimal amount;

int quantity; ```

The code does not explain meaning.

---

Better:

```java CustomerId customerId;

EmailAddress email;

Money amount;

Quantity quantity; ```

---

Good candidates:

## IDs

Instead of:

```
java String id;
```

Use:

```
java OrderId id;
```

Percentages

Instead of:

```
java double discount;
```

Use:

```
java Percentage discount;
```

Date ranges

Instead of:

```java LocalDate start;

LocalDate end; ```

Use:

```
java DateRange period;
```

---
```

Use value objects when:

- The concept has rules.
- The concept appears frequently.
- The meaning matters to the business.

---

9. Can a value object contain behavior?

Yes.

A value object should contain behavior related to its value.

Example:

Money:

```
java Money total = price.add(tax);
```

Not:

```
java MoneyCalculator.add(price,tax);
```

Email:

```
java email.normalize();
```

DateRange:

```
java dateRange.contains(date);
```

Good value object:

```java class Percentage {

private int value;

public Percentage increase(int amount){

}

} ```

---

Value objects are not just containers.

They are small domain objects.

---

# 10. Can a value object contain another value object?

Yes.

This is common.

Example:

Address:

```text Address

| | ├── Street | ├── City | └── PostalCode ```

PostalCode can itself be a value object.

---

Example:

Money:

```text Money

| | ├── Amount | └── Currency ```
```

Currency can be a value object.

Complex concepts can be composed from smaller concepts.

---

11. Should value objects be persisted separately?

Usually, no.

Most value objects are stored as part of their owning entity.

Example:

Customer:

```java class Customer {

CustomerId id;

Address address;

} ```

Database:

```text CUSTOMER_TABLE

id street city postal_code ```

Address does not need its own table.

---

However, sometimes they can have separate storage.

Example:

Large reusable address system:

```
text ADDRESS_TABLE
```

used by:

- Customer
- Supplier
- Warehouse

Then separate persistence may make sense.

Rule:

> Persistence decision depends on business needs, not because it is a value object.

---

12. What is the difference between Entity and Value Object?

| Entity | Value Object | | ---------- | ---------- | | Has identity | No identity | | Changes over time | Usually immutable | | Lifecycle exists | No lifecycle | | Equality by ID | Equality by value | | Controls state changes | Represents a value | | Example: Order | Example: Money |

Example:

Order

```text Order ID: 5001

Status: Created → Paid → Shipped ```

Same order over time.

Entity.

---
```

Money

`text 100 USD`

Always represents that value.

Value Object.

---

13. What are the performance/design costs of introducing many value objects?

Value objects are useful, but overusing them can create complexity.

Problems:

---

1. Too many classes

Example:

Creating:

```text FirstName

LastName

MiddleName

PhoneNumber

CountryCode

ZipCode ```

for everything may become unnecessary.

---

### 2. More mapping complexity

Example:

JPA mapping:

`java @Embeddable Money`

requires additional configuration.

---

### 3. Learning curve

New developers need to understand many domain concepts.

---

### 4. Object creation overhead

Creating thousands of tiny objects may have a small performance cost.

Usually insignificant in normal applications.

---

The key is balance.

Create value objects when they add:

- Meaning
- Safety
- Business rules

Do not create them only because DDD says so.

---
```

Important Examples

1. Money

```java Money {

amount;

currency;

add();

subtract();

} ```

Represents:

```
text A financial amount with rules.
```

---

## 2. EmailAddress

```java EmailAddress {

value;

validate();

} ```

Represents:

```
text A valid email identity.
```

---

3. Address

```text Address

Street

City

PostalCode ```

Represents:

```
text A location concept.
```

---

## 4. OrderId

```text OrderId

ORD-10001 ```

Represents:

```
text The identity value of an order.
```

---

5. CustomerId

```text CustomerId

CUS-5001 ```

Represents:

```
text The identity value of a customer.
```

---

## 6. DateRange

```text Start Date

End Date ```

Behavior:

```
java contains(date)
```

---

7. Percentage

```
text Discount = 20%
```

Rule:

```
text 0 <= value <= 100
```

---

8. Quantity

```
text Quantity = 5 items
```

Rules:

```
text Cannot be negative.
```

Core principle to remember:

> A Value Object represents a concept by what it is, not who it is.

A good value object:

```text ✓ Has no identity

✓ Is immutable

✓ Validates itself

✓ Contains meaningful behavior

✓ Makes the domain language clearer ```

The question is not:

> "Does this need a database table?"

The better question is:

> "Does this concept have its own meaning and rules in the business?"

---

$$\text{Aggregate}$$

---

# 7. Aggregate

This is one of the most important concepts in Domain-Driven Design because **aggregate design determines how your system protects business rules, manages consistency, handles transactions, and scales in distributed environments.**

Let's go deep.

---

# 1. What is an aggregate?

An **aggregate** is a cluster of related domain objects that are treated as **one consistency boundary**.

In simple words:

> An aggregate is a group of entities and value objects that must be controlled together to keep business rules valid.
```

An aggregate contains:

- One or more entities
- Zero or more value objects
- Business rules (invariants)
- One aggregate root

Example:

```text Order Aggregate

    Order
      |
      |

---

| | OrderItem ShippingAddress

      |
    Money

```

Here:

`text Order`

is not just a table.

It controls:

- Adding items
- Removing items
- Calculating total
- Changing status
- Cancelling order

---

2. Why do aggregates exist?

Aggregates exist because complex systems need a way to control **consistency and business rules**.

Without aggregates, any object could modify anything.

Example:

Without aggregate:

`java orderItem.setPrice(0);`

Anyone can change an order item.

Problem:

The order may become invalid.

With aggregate:

`java order.changeItemPrice(itemId, price);`

The Order controls the operation.

It can check:

```text Is the order still editable?

Is the user allowed?

Is the price valid? ```

---

The aggregate creates a protection boundary.

---

# 3. What problems do aggregates solve?

Aggregates solve several important problems.
```

1. Protect business rules

Example:

Rule:

> A paid order cannot be cancelled.

The Order aggregate protects this:

```
java order.cancel();
```

Inside:

```java if(status == PAID){

throw exception;

} ```

---

## 2. Control consistency

Example:

Order:

```
text Total = 100
```

Order items:

```text Item A = 60

Item B = 40 ```

The aggregate ensures:

```
text Total = Items Sum
```

---

3. Define transaction boundaries

An aggregate defines:

> What must change together in one operation.

Example:

Adding an item:

```text Order

+

OrderItem

+

Total calculation ```

should happen together.

---

## 4. Reduce coupling

Instead of many objects communicating:

```
text Customer OrderItem Payment Inventory
```

the aggregate creates controlled access.

---
```

4. What defines an aggregate boundary?

An aggregate boundary is defined by:

> Business consistency requirements.

Not:

- Database tables
- Object relationships
- ER diagrams

Wrong thinking:

"These tables have foreign keys, so they belong together."

Example:

Database:

```text Customer

Order

Payment

Address ```

Does not mean:

```
text One Aggregate
```

---

Correct question:

Ask:

> "Which objects must always remain consistent after a business operation?"

Those objects probably belong inside one aggregate.

---

Example:

Order:

```text Order

Order Items

Order Total ```

When adding an item:

```text Item added

Total updated ```

must happen together.

Therefore:

Same aggregate.

---

# 5. How is aggregate boundary related to transaction boundary?

They are strongly related.

An aggregate usually represents the smallest unit that should be updated in one transaction.

Example:

Order creation:
```

```text Create Order

Add Items

Calculate Total ```

One transaction:

```text BEGIN

Create Order

Add Items

Calculate Total

COMMIT ```

---

But:

Order + Payment

should usually not be one transaction.

Why?

Because:

`text Order`

and

`text Payment`

have different lifecycles.

Better:

```text Order Aggregate

    |
    |

OrderCreated Event

    ↓

Payment Aggregate ```

---

# 6. Which objects belong inside an aggregate?

Objects belong inside when:

## 1. They cannot exist meaningfully without the root

Example:

OrderItem.

Can an OrderItem exist without an Order?

Usually no.

Therefore:

```text Order

└── OrderItem

```

```

2. They participate in the same business rules

Example:

Order:

```text Item quantity

Price

Discount

Total ```

All affect order correctness.

---

### 3. They must change together

Example:

Changing quantity:

```text OrderItem quantity changes

↓

Order total changes ```

Same consistency boundary.

---

## 7. Which objects should remain outside?

Objects should stay outside when:

### 1. They have independent lifecycle

Example:

Customer.

A customer exists before and after orders.

Therefore:

```text Customer Aggregate

Order Aggregate ```

Separate.

---

### 2. They have different business rules

Example:

Payment:

```text Payment rules

Fraud checks

Refund rules ```

Different from Order rules.

---

### 3. They change independently

Example:

Inventory:

```
text Stock changes constantly
```

Order:

```
text Order lifecycle
```

Different reasons for change.

---

8. What does consistency boundary mean?

A consistency boundary defines:

> The area where business rules must always be correct immediately.

Example:

Order aggregate:

```text Order Total

Order Items

Discount ```

When order changes:

Everything must remain consistent.

---

Inside aggregate:

```
text Strong consistency
```

Outside aggregate:

```
text Eventual consistency
```

Example:

Inside:

```text Order

OrderItem

Total ```

Immediate consistency.

Outside:

```text Order

Payment ```

May become consistent later.

---

## 9. Why should aggregates generally be small?

Because large aggregates create problems.

A small aggregate:

```text Order

├── Items

└── Total ```

is manageable.

---

A large aggregate:

```text Customer

├── Orders

├── Payments

├── Addresses

├── Preferences

├── Reviews

├── Support Tickets ```

creates:

- Large transactions
- More locking
- More conflicts
- Poor scalability

---

DDD principle:

> Design aggregates around consistency, not around convenience.

---

# 10. Can an aggregate contain multiple entities?

Yes.

Example:

```text Order Aggregate

Order (Entity) | | OrderItem (Entity) | | ProductSnapshot (Value Object) ```

The aggregate root controls internal entities.

---

Outside world sees:

```
text Order
```

not:

```
text OrderItem
```

directly.

---

11. Can it contain value objects?

Yes.

Very commonly.

Example:

```text Order Aggregate

Order

| | ├── Money

├── ShippingAddress

└── Quantity ```

Value objects are usually owned by entities.

---

Example:

```java class Order {

private Money total;

private Address shippingAddress;

} ```

---

# 12. Can one aggregate directly reference another aggregate?

Usually no.

Example:

Bad:

```java class Order {

Customer customer;

} ```

Why?

Now:

```
text Order depends on entire Customer object
```

Creates coupling.

---

Better:

```java class Order {

CustomerId customerId;

} ```

Only reference identity.

---

# 13. Why is referencing another aggregate by ID usually preferred?

Because aggregates should be independent.

Example:

Order:

```
java CustomerId customerId;
```

Instead of:

```
java Customer customer;
```

Benefits:

1. Less coupling

Order does not know Customer internals.

---

2. Better scalability

No need to load huge object graphs.

---

3. Better distributed design

Different aggregates may live in different services.

Example:

```text Order Service
```

Customer Service ```

They communicate through:

```
text CustomerId + API/Event
```

---

14. What business rules should be enforced inside the aggregate?

Rules that protect the aggregate's consistency.

Example:

Order:

Rule:

Order cannot ship without payment.

Inside:

```
java order.ship();
```

checks:

```
text Payment status = PAID?
```

Order rules:

```text Cannot add item after shipment.

Cannot cancel completed order.

Total cannot be negative.

Quantity must be positive. ```

---

Do not put unrelated rules inside.

Example:

Order should not know:

```
text How payment gateway works.
```

---

# 15. What happens when aggregates become extremely large?

Large aggregates create serious problems.

---

## 1. Performance issues

Loading:

```text Customer

10000 Orders

50000 Items ```

is expensive.

---

## 2. Concurrency problems

Two users update the same aggregate.

Example:

```text User A updates address
```

User B places order ```

Conflict.

---

3. Database locking

Large aggregate means:

```text More rows locked

More waiting

Less throughput ```

---

### 4. Difficult deployment

The aggregate becomes a mini-monolith.

---

## 16. How does aggregate design affect concurrency?

Aggregate size directly affects concurrency.

Large aggregate:

```text Many users

↓

Same aggregate

↓

Conflicts ```

---

Small aggregate:

```text User A

Order 1

User B

Order 2 ```

Independent updates.

---

Example:

Bad:

```text Customer Aggregate

contains all Orders ```

Every order update locks customer.

---

Better:

```text Customer Aggregate

Order Aggregate ```

Independent.

---

## 17. How does aggregate design affect database locking?

Transactions usually lock aggregate data.
```

Large aggregate:

```text Update Customer

locks:

Customer

Orders

Addresses

Payments ```

Bad.

---

Small aggregate:

```text Update Order

locks:

Order only ```

Better.

---

Good aggregate design improves:

- Throughput
- Scalability
- Performance

---

# 18. How does aggregate design change in distributed systems?

Distributed systems cannot rely on one database transaction.

Therefore:

Aggregates become even more important.

Inside aggregate:

```
text Strong consistency
```

Between aggregates:

```
text Eventual consistency
```

Example:

Order:

```
text OrderCreated
```

Event:

```text Inventory Service

Payment Service

Shipping Service ```

Each aggregate reacts independently.

---

# 19. How do we maintain consistency between aggregates?

Through:

## 1. Domain Events

Example:
```

Order publishes:

```
text OrderPlaced
```

Inventory receives:

```
text Reserve Stock
```

---

2. Saga Pattern

A business process across multiple aggregates.

Example:

```text Order Created

↓

Reserve Inventory

↓

Charge Payment

↓

Ship Product ```

If payment fails:

```
text Release Inventory
```

---

### 3. Process Managers

Coordinate long-running workflows.

---

## 20. When do domain events become necessary?

Domain events become useful when:

- Something important happens.
- Other parts of the system need to react.
- Multiple aggregates are involved.

Example:

Order aggregate:

```
text OrderConfirmed
```

Other reactions:

```text Payment Service

Shipping Service

Notification Service ```

---

The Order does not directly call everyone.

It announces:

> "This business event happened."

---

# Example Analysis

### Order Aggregate

Given:

```text Order

├── OrderItem

├── ShippingAddress

└── Money ```

Which should be the aggregate root?

Answer:

```
text Order
```

Why?

Because:

## 1. Order has identity

Example:

```
text Order ID: 1001
```

It exists through time.

---

2. Order controls lifecycle

Order decides:

```text Created

Confirmed

Paid

Shipped

Cancelled ```

---

## 3. Order owns consistency rules

Example:

```text Total must match items.

Cannot ship empty order.

Cannot cancel shipped order. ```

---

## 4. Other objects depend on Order

OrderItem:

```
text belongs to an order
```

ShippingAddress:

```
text describes delivery for order
```

Money:

```
text describes order value
```

Final structure:

```text Order Aggregate

    Order
  (Aggregate Root)

        |
```

```

| |

OrderItem ShippingAddress

|
Money

```

External access:

```java order.addItem();

order.cancel();

order.ship(); ```

Not:

```
java orderItem.changeQuantity();
```

Core principle to remember:

> An aggregate is not a collection of related objects. It is a consistency boundary that protects business rules.

A good aggregate:

```text ✓ Has one aggregate root

✓ Protects invariants

✓ Controls state changes

✓ Is small

✓ Owns its data

✓ Communicates with other aggregates through IDs/events ```

The most important design question is:

> "What must always be consistent together?"

That answer usually reveals the aggregate boundary.

```

```

---

$$\text{Aggregate root}$$

---

# 8. Aggregate Root

Aggregate Root is one of the most important concepts in DDD because it defines **how an aggregate is controlled, protected, and accessed**.

A simple way to remember:

> The aggregate root is the gatekeeper of an aggregate.

Nobody outside the aggregate can directly modify the objects inside it. Everything must go through the root.

---

# 1. What is an aggregate root?

An **aggregate root** is the main entity inside an aggregate that controls access to all other objects inside that aggregate.

It is:

- An entity.
```

- The entry point of the aggregate.
- The owner of business rules.
- The guardian of consistency.

Example:

```text id="agg-root-1" Order Aggregate

    Order

(Aggregate Root)

        |
---------------
|             |

OrderItem ShippingAddress

      |
     Money

```

Here:

`text id="order-root" Order`

is the aggregate root.

External objects interact with:

`java order.cancel(); order.addItem(); order.confirm();`

They do not directly manipulate:

`java orderItem.changeQuantity();`

---

2. Why must every aggregate have a root?

Because without a root, there is no clear ownership or control.

Imagine:

```text id="no-root" Order

OrderItem

Address ```

Who controls the rules?

Who decides:

- Can an item be added?
- Can quantity change?
- Can the order be cancelled?
- Is the total valid?

Nobody.

The system becomes chaotic.

---

The root provides:

```text id="root-role" One entry point

    ↓

One place for rules

    ↓

One owner of consistency ```

---

Example:

Without root:

```
java orderItem.setPrice(0);
```

Possible.

Problem:

The order total becomes wrong.

With root:

```
java order.changeItemPrice(itemId, price);
```

Order checks:

```text id="order-check" Is modification allowed?

Is order editable?

Is price valid? ```

---

# 3. What responsibilities belong to the root?

An aggregate root has several responsibilities.

---

## 1. Protect business rules (invariants)

Example:

Order rule:

> A shipped order cannot be cancelled.

The root controls this:

```java public void cancel(){
if(status == SHIPPED){
    throw new OrderException();
}
status = CANCELLED;
} ```

---

## 2. Control access to internal objects

Example:

Bad:

```
java orderItem.setQuantity(10);
```

Good:

```
java order.changeQuantity(itemId,10);
```

The root decides whether the operation is allowed.

---

3. Maintain consistency

Example:

Order:

```text Items: Laptop = 1000 Mouse = 50

Total: 1050 ```
```

When an item changes:

`text Order | └── recalculates total`

The root keeps everything consistent.

---

4. Control lifecycle

Example:

Order lifecycle:

```text Created

↓

Confirmed

↓

Paid

↓

Shipped

↓

Delivered ```

The root manages these transitions.

---

## 4. Why should external objects interact through the root?

Because the root protects the internal state.

Think about a bank account.

Bad:

`java account.balance = -5000;`

Anyone can break the rules.

---

Better:

`java account.withdraw(amount);`

The account checks:

```text Is there enough balance?

Is withdrawal allowed?

Are limits exceeded? ```

---

The root acts like a security door:

```text Outside World

   |

   ↓

Aggregate Root

   |

   ↓

Internal Objects ```

---
```

Benefits:

- Prevents invalid states.
- Keeps business rules centralized.
- Reduces coupling.
- Makes changes safer.

---

5. How does the root protect invariants?

An invariant is a rule that must always remain true.

Example:

Order invariant:

> Order total must equal the sum of item prices.

Bad:

```
java orderItem.setPrice(500);
```

Now:

```text Items total = 500

Order total = 1000 ```

Invalid.

---

Root approach:

```
java order.updateItemPrice(itemId,500);
```

Inside:

```java public void updateItemPrice(){

item.changePrice();

recalculateTotal();

} ```

The root ensures:

```text Items

+

Total

stay consistent ```

---

Another example:

Bank Account:

Invariant:

```
text Balance cannot be negative.
```

Root:

```
java account.withdraw(500);
```

Checks before changing state.

---

6. Can internal entities be modified directly?

Normally, no.

Internal entities are controlled by the aggregate root.
```

Example:

```text id="internal-entity" Order Aggregate

Order

| └── OrderItem ```

Outside code should NOT do:

```
java orderItem.changeQuantity(5);
```

Because it bypasses Order rules.

---

Instead:

```
java order.changeItemQuantity(itemId,5);
```

The root decides:

- Is the order still active?
- Is quantity valid?
- Should total change?

Inside the aggregate, however:

Yes, internal entities can collaborate.

Example:

```
java Order | ↓ OrderItem.updateQuantity()
```

because the root controls that relationship.

---

7. Should repositories exist for every entity or only aggregate roots?

Usually:

> Repositories should exist only for aggregate roots.

Example:

Order aggregate:

```text Order Repository

↓

Stores Order ```

Not:

```
text OrderItem Repository
```

because OrderItem does not exist independently.

Why?

Because repositories represent aggregate access.

Example:

Good:

```
java orderRepository.findById(orderId);
```

Returns:

```
text Order Aggregate
```

including:

```
text OrderItems Address
```

Bad:

```
java orderItemRepository.findById(itemId);
```

Now you bypass Order rules.

Exception:

If an entity has its own lifecycle and can exist independently, it may become its own aggregate root.

---

8. Can an aggregate root reference another aggregate root?

Yes, but usually by ID.

Example:

Order needs Customer.

Bad:

```java class Order {

Customer customer;

} ```

Problem:

Order now depends on the entire Customer object.

---

Better:

```java class Order {

CustomerId customerId;

} ```

Now:

Order knows:

```
text Which customer?
```

but not:

```
text How customer works internally.
```

Example:

```text Order Aggregate

customerId = C123

Customer Aggregate

id = C123 ```

They communicate through identity.

---

# 9. How does an aggregate root control lifecycle?

The root defines valid state transitions.

Example:

Order:

```
text CREATED | ↓ CONFIRMED | ↓ PAID | ↓ SHIPPED
```
```

The root prevents invalid transitions.

Bad:

```
java order.status = "SHIPPED";
```

Possible problem:

Order was never paid.

Good:

```
java order.ship();
```

Inside:

```java public void ship(){

if(status != PAID){

    throw new Exception(
    "Order must be paid first");

}

status = SHIPPED;

} ```
```

The root becomes the owner of the lifecycle.

---

10. What makes a good aggregate root API?

A good aggregate root API expresses **business actions**, not data changes.

Bad API:

```java setStatus()

setAmount()

setType()

```

These expose internal details.

Good API:

```java confirmOrder();

cancelOrder();

shipOrder();

addItem();

removeItem();

applyDiscount(); ```

Why?

Because these represent business intentions.

---

Compare:

Bad:

```
java order.setStatus(PAID);
```
```

Question:

- Is payment actually completed?
- Was payment verified?
- Are rules checked?

Good:

```
java order.markAsPaid(paymentReference);
```

The root can validate everything.

---

11. Why is this:

```
java order.setStatus("PAID");
```

often worse than:

```
java order.markAsPaid();
```

Let's compare.

---

setStatus()

This says:

> Change a piece of data.

It exposes implementation.

Anyone can do:

```
java order.setStatus("PAID");
```

Even when:

- Payment failed.
- Payment was cancelled.
- Fraud check failed.

---

markAsPaid()

This says:

> A business event happened.

The method can contain rules:

```java public void markAsPaid(){

if(payment == null){

    throw new Exception();

}

status = PAID;

raiseEvent(
   new OrderPaid()
);

} ```
```

The difference:

```text setStatus()

Data manipulation

markAsPaid()

Business behavior ```
```

DDD prefers business behavior.

12. How does aggregate-root design prevent invalid states?

By forcing all changes through controlled operations.

Example:

Without root protection:

```
java orderItem.quantity = -5;
```

Invalid state.

With root:

```
java order.changeQuantity(-5);
```

Inside:

```java if(quantity < 0){

throw InvalidQuantityException;

} ```

---

Another example:

Order:

Allowed:

```
text CREATED → CANCELLED
```

Not allowed:

```
text SHIPPED → CREATED
```

The root prevents illegal transitions.

The goal:

> Make illegal states impossible or difficult to create.

Complete Example: Order Aggregate Root

```text Order Aggregate

    Order

(Aggregate Root)

    |
    |
```

| |

OrderItem ShippingAddress

```
   |
  Money
```

```

External usage:

```java Order order = orderRepository.find(orderId);

order.addItem(product, quantity);

order.confirm();

order.markAsPaid();
```

order.ship(); ```

Never:

```java orderItem.setQuantity();

address.setCity();

order.status = "PAID"; ```

---

# Core principle to remember:

> The aggregate root is the only door into an aggregate.

A good aggregate root:

``` ✓ Owns the aggregate lifecycle

✓ Protects invariants

✓ Controls internal entities

✓ Exposes business actions

✓ Prevents invalid states

✓ Is the only repository entry point ```

The most important question when designing a root is:

> "What business decisions must this object control to keep the aggregate correct?"

$$\text{Invariant}$$

---

9. Invariant

Invariant is one of the deepest concepts in Domain-Driven Design because it connects:

- Entity
- Value Object
- Aggregate
- Aggregate Root
- Transaction
- Consistency
- Distributed Systems

A simple definition:

> An invariant is a business rule that must always remain true for the domain model to be valid.

It defines what states are allowed and what states are forbidden.

---

1. What is an invariant?

An **invariant** is a condition that must always be maintained by the domain.

It protects the correctness of the business.

Example:

Order Domain

Rule:

> Order total cannot be negative.

Invariant:

```
text Order.total >= 0
```

Shipping Domain

Rule:

> A shipped order cannot be cancelled.

Invariant:

```text Order Status = SHIPPED

cannot transition to

CANCELLED ```

---

### Banking Domain

Rule:

> Account balance cannot exceed allowed credit limit.

Invariant:

```
text Balance >= -CreditLimit
```

---

An invariant answers:

> "What must always be true for this business object to make sense?"

---

## 2. What is the difference between an invariant and validation?

They are related but different.

### Validation

Validation checks whether input is acceptable.

Example:

User enters email:

```
text abc@test.com
```

Validation:

```
text Is this a valid email format?
```

Invariant

An invariant protects a business rule after the object exists.

Example:

Order:

```
text Cannot ship without payment.
```

This is not just input validation.

It is a rule about business state.

Comparison:

| Validation | Invariant | | ---------- | ---------- | | Checks input | Protects business correctness | | Usually happens at boundaries | Lives inside domain | | Often technical | Business-focused | | Example: email format | Example: paid order cannot be cancelled |

Example:
```

Creating an order:

Validation:

```
text Quantity must be positive.
```

Invariant:

```
text Total must equal sum of items.
```

---

3. What is the difference between validation and business rules?

Validation:

> Is this data acceptable?

Business rule:

> Is this action allowed according to the business?

Example:

Customer registration:

Validation:

```
text Email format is correct. Password length is enough.
```

Business rules:

```text A blocked customer cannot place orders.

A premium customer gets a discount.

A cancelled subscription cannot be renewed immediately. ```

---

Technical validation:

```
text String length < 255
```

Business rule:

```
text Customer cannot buy more than allowed quantity.
```

---

4. Who is responsible for protecting invariants?

The responsibility depends on where the invariant belongs.

Entity Invariants

Protected by the entity.

Example:

Account:

```
text Balance cannot become negative.
```

Code:

```
java account.withdraw(amount);
```

Inside Account:

```java if(balance < amount){

throw Exception;

} ```

### Value Object Invariants

Protected by the value object.

Example:

Money:

```
text Amount cannot be negative.
```

```
java new Money(-100,"USD");
```

Should fail.

Aggregate Invariants

Protected by the aggregate root.

Example:

Order:

```
text Cannot ship unpaid order.
```

Controlled by:

```
java order.ship();
```

The rule:

> The object that owns the rule should protect the rule.

---

5. Should invariants always be true?

Yes.

That is the purpose of an invariant.

An invariant describes a state that should never be invalid.

Example:

Account:

Allowed:

```
text Balance = 1000
```

Allowed:

```
text Balance = -500 Credit limit = 1000
```

Not allowed:

```
text Balance = -5000 Credit limit = 1000
```

The domain model should prevent invalid states from existing.

However, in distributed systems, some invariants may temporarily be violated between different aggregates.

This leads to eventual consistency.

---

6. Can an aggregate temporarily enter an invalid state?

Inside one aggregate:

Usually no.

An aggregate should never commit an invalid state.

Example:

Order:

```text Order Item added

but

Total not updated ```

should never be stored.

The transaction should:

```text Update item

+

Update total

=

Commit ```

together.

---

However, across aggregates:

Temporary inconsistency may happen.

Example:

Order:

```
text Order Created
```

Inventory:

```
text Stock not reserved yet
```

For a short time:

```text Order exists

Inventory not updated ```

This is acceptable in distributed systems.

---

# 7. What is a transactional invariant?

A **transactional invariant** is a rule that must remain true within a single transaction.

Usually, it belongs inside one aggregate.

Example:

Order:

Before:

```text Items: A = $50

Total: $50 ```

Operation:

Add item:

```
text B = $20
```

After:

```text Items: A = $50 B = $20
```

Total: $70 ```

The change must happen atomically.

Transaction:

```text BEGIN

Add Item

Calculate Total

COMMIT ```

---

Transactional invariant:

```
text Order total must equal item sum.
```

---

# 8. What is a cross-aggregate invariant?

A cross-aggregate invariant is a business rule involving multiple aggregates.

Example:

Order:

```
text Order must have available inventory.
```

Two aggregates:

```text Order Aggregate

Inventory Aggregate ```

---

Another example:

Bank:

```
text Total money transferred out cannot exceed account balance.
```

Involves:

```text Sender Account

Receiver Account ```

---

The problem:

Aggregates are independent.

One transaction cannot easily update both.

---

# 9. How are cross-service invariants handled?

In microservices, cross-service rules are usually handled through:

- Domain events
- Saga pattern
- Process managers
- Eventual consistency

---

Example:

Order process:

Step 1:

Order Service:

```
text OrderCreated
```
```

publishes event.

Step 2:

Inventory Service:

```
text Reserve Inventory
```

Step 3:

Payment Service:

```
text Process Payment
```

If payment fails:

Compensating action:

```
text Release Inventory Cancel Order
```

Instead of:

```
text One giant transaction
```

we use:

```
text Business workflow
```

---

10. What happens when strong consistency isn't possible?

Distributed systems often cannot provide immediate consistency.

Example:

Amazon-like system:

Customer places order:

```
text Order Service
```

Inventory:

```
text Inventory Service
```

Payment:

```
text Payment Service
```

Different databases.

A single transaction:

```text BEGIN

Update Order DB

Update Inventory DB

Update Payment DB

COMMIT ```

is not practical.

---

Instead:

Use eventual consistency.

Meaning:
```

The system may be temporarily inconsistent but will become correct later.

Example:

Immediately:

```text Order = Created

Inventory = Pending ```

After seconds:

```
text Inventory = Reserved
```

---

# 11. How does eventual consistency affect invariants?

It changes how we think about invariants.

Inside aggregate:

```
text Strong consistency
```

Example:

Order total.

Between aggregates:

```
text Eventual consistency
```

Example:

Order + Inventory.

Example:

Invariant:

Old thinking:

Order must always have inventory.

Distributed reality:

Order must eventually have confirmed inventory or be cancelled.

The system handles temporary states:

```text Pending Reservation

↓

Confirmed

or

Failed ```

---

# 12. Should database constraints also enforce invariants?

Yes, but only as a safety layer.

Database constraints are useful for:

- NOT NULL
- UNIQUE
- Foreign keys
- Check constraints

Example:

Database:

```
sql CHECK(balance >= -credit_limit)
```

Good.

But database constraints cannot replace domain logic.

Example:

Business rule:

> Paid customers get 10% discount.

A database cannot understand this.

Best approach:

```text Domain Model
    +
```

Database Protection ```

Domain:

```
java customer.applyDiscount();
```

Database:

```
sql CHECK(price >= 0)
```

Different responsibilities.

---

13. What happens when business invariants exist only in application services?

This creates weak domain models.

Example:

Bad design:

```java OrderService {
if(order.status == PAID){
throw Exception;
}
order.cancel();
} ```

Problem:

Another place may do:

```java AnotherService {
order.status = CANCELLED;
} ```

Now rules are duplicated.

---

Problems:

### 1. Rules scattered everywhere

Nobody knows the complete business logic.

---

### 2. Inconsistent behavior

Different services may apply different rules.

---

### 3. Hard maintenance

Business changes require searching everywhere.

---

Better:

Move rule into domain:

```
java order.cancel();
```

Inside:

```java if(status == PAID){

throw Exception;

} ```

Now one owner.

---

# Example Analysis

## Rule 1:

```
text Order total cannot be negative.
```

Where should it live?

Answer:

#### Money Value Object

Because money owns money rules.

Example:

```java class Money {

```
Money(amount){

    if(amount < 0)
        throw Exception;

}
```

} ```

Rule 2:

```
text A shipped order cannot be cancelled.
```

Where should it live?

Answer:

Order Aggregate Root

Because Order controls its lifecycle.

Example:

```
java order.cancel();
```

Inside:

```java if(status == SHIPPED){

throw Exception;

} ```

---

# Rule 3:

```
text Account balance must never violate credit limit.
```

Where should it live?

Answer:

### Account Entity / Aggregate Root

Because Account owns:

- Balance
- Credit rules
- Withdrawal behavior

Example:

```
java account.withdraw(amount);
```

Final Mapping

```text Rule | | ↓

Money amount rules

    → Value Object

Order lifecycle rules

    → Order Aggregate Root

Account balance rules

    → Account Aggregate Root

Order + Inventory consistency

    → Domain Events / Saga

```

Core principle to remember:

> An invariant is a rule that protects the truth of your domain.

A good DDD design asks:

```text Who owns this rule?

Where can this rule be violated?

Which boundary protects it? ```

The strongest systems are built by placing each invariant in the correct owner:

```
text Value Object ↓ Entity ↓ Aggregate Root ↓ Domain Events ↓ Distributed Workflow
```

That is how a domain model stays correct as the system grows.

$$\text{Repository}$$

---

10. Repository

Repository is the bridge between the **Domain Model** and **Persistence Infrastructure**.

A simple way to remember:

> A repository provides access to domain objects without exposing how those objects are stored.

The domain should think:

> "I need an Order."

Not:

> "I need to write SQL, open a connection, and query a database."

---

1. What is a repository in DDD?

A **Repository** is an abstraction that provides access to aggregate roots and hides persistence details from the domain.

It behaves like a collection of domain objects.

Example:

Instead of:

```
sql SELECT * FROM orders WHERE id = 1001;
```

The application uses:

```
java Order order = orderRepository.findById(orderId);
```

The caller does not know:

- Which database is used.
- How the query works.
- How mapping happens.

Example:

```java public interface OrderRepository {

Order findById(OrderId id);

void save(Order order);

} ```

Implementation:

```java public class JpaOrderRepository implements OrderRepository {

} ```

The domain only knows the interface.

---

# 2. Why does repository abstraction exist?

Because the domain should not depend on infrastructure.

Without repository:

```java class OrderService {

EntityManager entityManager;

Order findOrder(){
    return entityManager.find(Order.class,id);
}

} ```

Problem:
```

Now business logic knows:

- JPA
- Database
- ORM details

The domain becomes coupled to technology.

With repository:

```
java Order order = orderRepository.findById(orderId);
```

The domain only knows:

```
text "I need an Order."
```

Benefits:

1. Separation of concerns

Domain:

```
text Business rules
```

Infrastructure:

```
text Database operations
```

---

2. Easier testing

You can replace:

```
text Database Repository
```

with:

```
text Fake Repository
```

during tests.

---

3. Easier technology changes

Today:

```
text PostgreSQL
```

Tomorrow:

```
text MongoDB
```

The domain remains unchanged.

---

3. What should a repository represent?

A repository should represent a collection of **domain aggregates**.

Example:

```
java OrderRepository
```

represents:

```
text Collection of Orders
```

Conceptually:

```java orders.add(order);

orders.find(orderId); ```

---
```

It should not represent:

- Database tables
- SQL queries
- Technical storage

Bad thinking:

```
text CustomerTableRepository OrderTableRepository
```

Better:

```
text CustomerRepository OrderRepository
```

Because the repository speaks the domain language.

---

4. Should every entity have a repository?

No.

This is a common mistake.

Repositories usually exist only for **aggregate roots**.

Example:

Order Aggregate:

```text Order (Aggregate Root)

| | └── OrderItem ```

Repository:

```
java OrderRepository
```

Not:

```
java OrderItemRepository
```

Why?

Because OrderItem does not have an independent lifecycle.

You do not normally ask:

```
java orderItemRepository.findById(itemId);
```

because that bypasses Order rules.

Instead:

```java Order order = orderRepository.findById(orderId);

order.removeItem(itemId); ```

---

An entity gets its own repository only if it becomes its own aggregate root.

---

# 5. Why are repositories usually defined around aggregate roots?

Because aggregate roots control consistency.

Example:

Order:

```text Order

| ├── OrderItem

└── ShippingAddress ```
```

The repository loads the complete aggregate:

```text Order

    • Items

    • Address ```

The application receives a valid business object.

---

If you create repositories for internal entities:

Example:

`java OrderItemRepository`

you can accidentally do:

`java orderItem.changePrice();`

without updating:

`text Order total`

The aggregate boundary is broken.

---

Rule:

> The repository is the gateway to the aggregate root.

---

# 6. Should repository interfaces belong to domain or infrastructure?

In DDD, the **repository interface belongs to the domain layer**.

Example:

Domain:

```java public interface OrderRepository {

Order findById(OrderId id);

void save(Order order);

} ```

Infrastructure:

```java @Repository public class JpaOrderRepository implements OrderRepository {

} ```

---

Why?

Because the domain defines what it needs.

The infrastructure provides the implementation.

This follows the Dependency Inversion Principle.

---

Dependency direction:

```text Infrastructure

    ↓ implements

Domain Interface ```

Not:

```text Domain

    ↓ depends on
```

Infrastructure ```

---

7. What operations should repositories expose?

Repositories should expose meaningful persistence operations.

Common operations:

Find

```
java Order findById(OrderId id);
```

Save

```
java void save(Order order);
```

Remove

```
java void delete(Order order);
```

Query by domain concepts

Example:

```
java List<Order> findPendingOrders(CustomerId customerId);
```

Good repository methods speak the domain language.

Example:

Good:

```
java findActiveSubscriptions();
```

Bad:

```
java findByStatus("ACTIVE");
```

The first expresses business meaning.

---

8. Should repositories expose generic CRUD operations?

Usually no.

Generic CRUD can leak technical thinking into the domain.

Example:

```java create()

read()

update()

delete() ```

These are database operations.

The domain thinks in business actions.

---

Bad:

```
java customerRepository.update(customer);
```

What does update mean?
```

Better:

```
java customerRepository.save(customer);
```

or:

```
java customerRepository.deactivate(customer);
```

depending on the domain.

Repositories should not become database wrappers.

---

9. Is `save()` always enough?

Not always.

For simple systems:

```
java repository.save(entity);
```

may be enough.

But complex domains may need more expressive operations.

Example:

Subscription:

Instead of:

```
java subscriptionRepository.save(subscription);
```

you may need:

```
java subscriptionRepository.findActiveSubscription(customerId);
```

or:

```
java subscriptionRepository.existsActivePlan(customerId);
```

The repository should expose domain-required operations.

Not every database operation.

---

10. Should repositories return domain entities?

Yes.

Repositories should return domain objects.

Example:

Good:

```
java Order order = orderRepository.findById(id);
```

Returns:

```
text Order Domain Object
```

Not:

```
java OrderDTO
```

or:

```
java OrderEntity
```

Why?

Because the application needs behavior.

Example:

```
java order.cancel();
```

A DTO cannot do this.

---

11. Should repositories return DTOs?

Usually no.

DTOs belong to communication boundaries.

Examples:

- REST API
- Message queues
- External integrations

Flow:

```text Database

    ↓

Repository

    ↓

Domain Entity

    ↓

Application Service

    ↓

DTO

    ↓

API Response ```

---

Do not make:

```
text Repository → DTO → Domain
```

because the domain becomes disconnected from persistence.

---

# 12. Should domain logic exist inside repositories?

No.

Repositories should not contain business rules.

Bad:

```java class OrderRepository {

public void save(Order order){

    if(order.total > 10000){

        applyDiscount();

    }

}

} ```

Why bad?

The repository now owns business decisions.

---

Repository responsibility:

```
text Store and retrieve objects
```

Domain responsibility:

```
text Decide business behavior
```

Correct:

```java order.applyDiscount();

orderRepository.save(order); ```
```

---

13. What is the difference between repository and DAO?

They look similar but have different purposes.

DAO (Data Access Object)

A technical pattern.

Concern:

> How do I access database data?

Example:

```
java UserDAO.findUserById();
```

It usually works with:

- SQL
- Tables
- Rows

Repository

A domain pattern.

Concern:

> How do I access domain objects?

Example:

```
java customerRepository.findPremiumCustomers();
```

It works with:

- Entities
- Aggregates
- Business concepts

Comparison:

| DAO | Repository | | ---------- | ---------- | | Technical abstraction | Domain abstraction | | Database focused | Business focused | | Returns data | Returns domain objects | | Table oriented | Aggregate oriented |

---

14. What is the relationship between Spring Data Repository and DDD Repository?

They are related but not identical.

Spring Data:

```
java JpaRepository<OrderEntity, Long>
```

is a technical repository.

Example:

```java public interface OrderJpaRepository extends JpaRepository{

} ```

It provides:

- save()
- findById()
- delete()
- CRUD operations

---

DDD Repository:

```java public interface OrderRepository {

```
Order findById(OrderId id);

void save(Order order);
```

} ```

It represents domain needs.

They can be connected:

```text DDD Repository

    ↓

Adapter

    ↓

Spring Data Repository

    ↓

Database ```

---

Do not blindly expose Spring Data directly as your domain repository.

---

# 15. How can JPA accidentally influence domain modeling?

JPA can push developers toward database-first thinking.

Example:

Developer sees tables:

```text ORDER

ORDER_ITEM

CUSTOMER ```

Creates:

```java @Entity class OrderEntity {

@OneToMany List items;

} ```

Then the domain becomes shaped by:

- Tables
- Foreign keys
- ORM relationships

instead of business rules.

---
```

Problems:

- Huge entities
- Too many relationships
- Anemic models
- Lazy loading problems

DDD question:

Not:

> "How do I map this table?"

Instead:

> "What business concept exists here?"

---

16. What happens when lazy loading leaks into domain logic?

Lazy loading is an ORM feature.

Example:

```
java order.getItems();
```

Hibernate suddenly queries:

```
sql SELECT * FROM order_items;
```

inside business logic.

Problems:

---

1. Hidden database calls

Developer thinks:

```
java calculateTotal();
```

But internally:

```
text Database query happens
```

---

2. Performance problems

Example:

N+1 query problem.

```text Load 100 Orders

↓

Each loads Items separately

↓

101 database queries ```

---

## 3. Domain depends on ORM behavior

The domain should not care about:

```
text Hibernate Session Persistence Context Lazy Proxy
```

---

Better:

Load the aggregate intentionally.
```

Example:

```
java Order order = orderRepository.findCompleteOrder(orderId);
```

Then:

```
java order.calculateTotal();
```

Complete Example

Order Aggregate

```text Order Aggregate
    Order
```

(Aggregate Root)

```
     |
```

OrderItem

ShippingAddress

Money ```

Repository:

```
java OrderRepository
```

Usage:

```java Order order = orderRepository.findById(orderId);

order.addItem(product,quantity);

order.confirm();

orderRepository.save(order); ```

The application does not know:

- SQL
- JPA
- Database tables

---

# Core principle to remember:

> A repository is not a database wrapper. It is a domain gateway for retrieving and storing aggregates.

A good repository:

```text ✓ Works with aggregate roots

✓ Returns domain objects

✓ Hides persistence details

✓ Does not contain business rules

✓ Uses domain language

✓ Keeps the domain independent ```

The key question:

> "If tomorrow I replace my database, will my domain model still make sense?"

If the answer is yes, your repository design is probably healthy.

---

$$\text{Domain Service}$$

# 12. Application Service

The **Application Service** is the layer that coordinates the use cases of the system.

A simple way to remember:

> The application service tells the domain what needs to happen, but the domain decides how business rules are executed.

It is the **orchestrator**, not the business expert.

---

# 1. What is an application service?

An **Application Service** is a service in the application layer that coordinates a complete business use case.

It controls:

- The flow of an operation.
- Calling domain objects.
- Managing transactions.
- Communicating with external systems.
- Returning results.

It does not contain core business decisions.

---

Example:

Use case:

> Customer places an order.

Application Service:

```java public class PlaceOrderService {

public void placeOrder(Command command){

    Customer customer =
        customerRepository.findById(command.customerId);


    Order order =
        Order.create(customer);


    order.addItems(command.items);


    orderRepository.save(order);

}

} ```
```

Notice:

The service coordinates.

But the Order decides:

- Whether items can be added.
- Whether the order is valid.
- How totals are calculated.

---

2. What responsibility does the application layer have?

The application layer is responsible for **application workflow**.

It answers:

> "What steps should happen to complete this use case?"

It handles:

1. Use case coordination

Example:

```
text Place Order Cancel Order Approve Loan Register Customer
```

---

2. Calling domain objects

Example:

```
java order.confirm();
```

---

3. Managing transactions

Example:

```
java @Transactional placeOrder();
```

---

4. Calling repositories

Example:

```
java orderRepository.findById(id);
```

---

5. Calling external systems

Example:

```
text Payment Gateway Email Service Shipping API
```

It connects the outside world with the domain.

---

3. Should application services contain business rules?

Generally, no.

Business rules belong inside:

- Entities
- Value Objects
- Aggregates
- Domain Services

Bad:

```java public void cancelOrder(Order order){

if(order.status.equals("SHIPPED")){

    throw Exception();

}

order.status="CANCELLED";

} ```

Problem:

The application service knows Order rules.

---

Better:

```java public void cancelOrder(OrderId id){

Order order =
    orderRepository.findById(id);


order.cancel();
```
```

```
orderRepository.save(order);
```

} ```

The decision belongs to Order.

Application service says:

> "Cancel this order."

Domain says:

> "Can this order be cancelled?"

---

4. What does orchestration mean?

Orchestration means coordinating multiple steps and objects to complete a business operation.

Example:

Customer checkout:

```text 1. Load Customer

1. Create Order
2. Reserve Inventory
3. Process Payment
4. Save Order
5. Publish Event ```

The application service coordinates this flow.

---

Example:

```
java checkoutService.checkout(cartId);
```

Internally:

```text Checkout Service
    |
    |
    ├── Cart Aggregate
    |
    ├── Order Aggregate
    |
    ├── Payment Service
    |
    └── Notification Service
```

The application service is like a conductor.

The musicians (domain objects) perform the actual work.

---

5. How should application services coordinate aggregates?

Aggregates should not directly control each other.

Application service coordinates them.

Example:

Order + Payment:

```text Order Aggregate

Payment Aggregate ```

Application Service:

```java public void completeOrder(OrderId id){

```
Order order =
   orderRepository.findById(id);

Payment payment =
   paymentService.pay(order);

order.markAsPaid();

orderRepository.save(order);
```

} ```

---

The service coordinates:

- Loading aggregates.
- Calling behavior.
- Saving changes.

---

But it should not do:

```
java order.status = PAID;
```

because that bypasses the aggregate.

---

6. Should they manage transactions?

Yes.

Application services are usually the correct place for transaction boundaries.

Example:

Spring:

```java @Transactional public void placeOrder(){

} ```

Why?

Because the application service represents a complete use case.

Example:

```text Place Order

BEGIN TRANSACTION

Create Order

Add Items

Save Order

COMMIT ```

---

The domain should not know about:

```
text Database Transaction Spring @Transactional Hibernate Session
```

---

# 7. Should they call repositories?

Yes.

Application services commonly use repositories.
```

Example:

```java public void updateOrder(OrderId id){

Order order =
    orderRepository.findById(id);

order.confirm();

orderRepository.save(order);

} ```

---

The flow:

```text Application Service

    ↓

Repository

    ↓

Aggregate Root

    ↓

Business Logic ```

---

Repositories retrieve and store.

They do not make decisions.

---

# 8. Should they call external APIs?

Yes.

This is one of their responsibilities.

Examples:

- Payment gateway
- Email provider
- Shipping API
- Identity provider

Example:

```java public void processPayment(OrderId id){

Order order =
   orderRepository.findById(id);

paymentGateway.charge(order.amount());

order.markAsPaid();

} ```

---

But:

The application service coordinates.

The domain decides.

---

Bad:

```java if(paymentResponse.success){

order.status="PAID";

} ```

Better:

```
java order.markAsPaid();
```

---

9. Should they publish events?

Yes, often.

After a successful business operation, application services may publish events.

Example:

```java public void createOrder(){

Order order =
   Order.create();


orderRepository.save(order);


eventPublisher.publish(
   new OrderCreated(order.id())
);

} ```
```

Events allow other parts of the system to react.

Example:

```text OrderCreated

   ↓

Inventory Service

   ↓

Notification Service

   ↓

Analytics Service ```
```

However, in advanced designs, domain events are usually created inside the domain model.

The application layer dispatches them.

---

10. Should they perform authorization?

Yes, but carefully.

Authorization is usually application-level concern.

Example:

```java public void cancelOrder( User user, OrderId orderId ){

authorization.check(
    user,
    "CANCEL_ORDER"
);


order.cancel();

} ```
```

But business rules remain in the domain.

Example:

Authorization:

> "Can this user cancel orders?"

Application layer.

Business rule:

> "A shipped order cannot be cancelled."

Domain layer.

Difference:

```text Authorization:

Who is allowed?

Business Rule:

What is allowed? ```

---

# 11. What is the difference between controller and application service?

They are different layers.

---

## Controller

Responsible for:

- HTTP handling
- Request parsing
- Response formatting

Example:

```java @PostMapping("/orders") public ResponseEntity createOrder(){

} ```

It knows:

```
text HTTP JSON Status Codes
```

Application Service

Responsible for:

- Use case execution
- Workflow coordination

Example:

```
java orderApplicationService.createOrder(command);
```

Flow:

```text HTTP Request

↓

Controller

↓

Application Service

↓

Domain Model

↓
```

Repository

↓

Database ```

Controller should be thin.

---

12. What is the difference between domain service and application service?

This is a very important distinction.

Application Service

Question:

> "How do we execute this use case?"

Example:

```
java placeOrder();
```

Responsibilities:

- Workflow
- Coordination
- Transactions
- Security
- External communication

Domain Service

Question:

> "What business logic does not naturally belong to one object?"

Example:

Currency conversion:

```
java currencyExchangeService.convert();
```

or:

Complex pricing:

```
java pricingService.calculateDiscount();
```

Comparison:

| Application Service | Domain Service | | ---------- | ---------- | | Application layer | Domain layer | | Coordinates actions | Contains business logic | | Calls repositories | Works with domain concepts | | Handles workflow | Makes domain decisions |

Example:

Application Service:

```
java checkoutService.checkout();
```

Domain Service:

```
java discountCalculator.calculate();
```

---

13. What does a thin application service look like?

A thin application service:

- Coordinates.
- Delegates.
- Does not make business decisions.

Example:

```java @Service class OrderApplicationService {

private final OrderRepository repository;

@Transactional public void cancelOrder(OrderId id){

Order order =
    repository.findById(id);

order.cancel();

repository.save(order);

}

} ```

Notice:

The service does not know:

```
text Why cancellation is allowed.
```

The Order knows.

---

# 14. What does an overly intelligent application service look like?

An overly intelligent application service contains domain logic.

Example:

```java public void checkout(){

if(customer.isPremium()){

    discount = 20;

}

if(order.total > 1000){

    shippingFree=true;

}

order.status="CONFIRMED";

} ```

Problems:

- Business rules are outside the domain.
- Logic becomes scattered.
- Domain objects become simple data holders.
- Difficult testing.

This creates an **anemic domain model**.

---

The better design:

```java order.confirm();

order.calculateDiscount();

order.applyShippingRules(); ```

---
```

15. How does the application service fit with Clean/Hexagonal Architecture?

Application Service sits between external systems and the domain.

Architecture:

```text Outside World

                |
                ↓

          Controller
          (Adapter)

                |
                ↓

    Application Service
      (Use Cases)

                |
                ↓

          Domain Model
    (Entities, Aggregates)

                |
                ↓

    Repository Interface
         (Port)

                |
                ↓

      Infrastructure
   (Database, APIs, Kafka)

```

In Hexagonal Architecture:

Application Service = Application Core

It defines:

- What the system can do.
- Available use cases.

Domain Model = Business Core

It defines:

- Business rules.
- Business behavior.

Infrastructure = External Details

It defines:

- Database
- Messaging
- Frameworks

Complete Example

Use case:

"Cancel Order"

Controller:

```java @PostMapping("/orders/{id}/cancel") public void cancel(){

orderService.cancelOrder(id);

} ```

---
```

Application Service:

```java @Transactional public void cancelOrder(OrderId id){

Order order =
    repository.findById(id);

order.cancel();

repository.save(order);

} ```

---

Domain:

```java class Order {

public void cancel(){

if(status == SHIPPED){

    throw Exception();

}

status = CANCELLED;

}

} ```

---

Repository:

```java interface OrderRepository {

Order findById(OrderId id);

void save(Order order);

} ```

---

Infrastructure:

```
text JPA PostgreSQL Kafka
```

---

# Core principle to remember:

> The application service coordinates the business process, but the domain model owns the business rules.

A healthy architecture looks like:

```
text Controller ↓ Application Service ↓ Domain Model ↓ Repository Interface ↓ Infrastructure
```

The application service asks:

> "What needs to happen?"

The domain model answers:

> "How can this happen correctly according to business rules?"

$$\text{Domain Event}$$

---

13. Domain Event

Domain Event is the concept where **Domain-Driven Design connects with Distributed Systems**.
```

It introduces a different way of communication:

Instead of one object directly calling another:

```
text Order Service | ↓ Payment Service | ↓ Shipping Service
```

The domain says:

> "Something important happened."

Other parts of the system decide whether they need to react.

A simple definition:

> A domain event represents a meaningful business fact that has already happened.

The key word is:

Happened

---

1. What is a domain event?

A **domain event** is an object that represents a significant occurrence in the business domain.

It describes:

- Something that happened.
- Something important to the business.
- Something other parts of the system may care about.

Example:

```
text OrderPlaced
```

means:

> A customer successfully placed an order.

Other examples:

```text PaymentCompleted

CustomerRegistered

OrderCancelled

SubscriptionRenewed

LoanApproved ```
```

A domain event is not an instruction.

It is a fact.

Compare:

Command:

```
text PlaceOrder
```

Meaning:

> Please do this.

Event:

```
text OrderPlaced
```

Meaning:

This already happened.

---

2. What does a domain event represent?

A domain event represents a **business state change**.

Example:

Before:

```text Order

Status: CREATED ```

Action:

```
text Customer confirms order
```

After:

```text Order

Status: CONFIRMED ```

The event:

```
text OrderConfirmed
```

represents this transition.

A domain event captures:

```text Business fact

+ Time

+ Relevant information ```

---

Example:

```
json { "eventType": "OrderPlaced", "orderId": "ORD-1001", "customerId": "CUS-500", "amount": 250 }
```

Meaning:

"The business fact happened."

---

## 3. Why should domain events normally use past tense?

Because events represent something that already happened.

They are facts.

---

Correct:

```text OrderPlaced

PaymentCompleted

CustomerRegistered

OrderCancelled ```

Meaning:

```
text Something happened.
```

Incorrect:

```text PlaceOrder
```

CompletePayment

RegisterCustomer ```

These are commands.

They represent requests.

Comparison:

| Command | Event | | ---------- | ---------- | | Request | Fact | | Future action | Past occurrence | | "Do this" | "This happened" |

Example:

Command:

```
text ApproveLoan
```

Someone asks:

> Approve this loan.

After successful approval:

Event:

```
text LoanApproved
```

Meaning:

> The loan has been approved.

---

4. When should an aggregate generate a domain event?

An aggregate should generate an event when an important business fact occurs.

Not every database change needs an event.

Good examples:

Order:

```
text OrderPlaced OrderCancelled OrderShipped
```

Payment:

```
text PaymentCompleted PaymentFailed RefundIssued
```

Customer:

```
text CustomerRegistered CustomerUpgraded
```

Bad examples:

```
text OrderNameChanged OrderFieldUpdated OrderRowModified
```

These are technical changes, not business events.

A good question:

> "Would a business person care that this happened?"

If yes, it may deserve a domain event.

---

5. Who publishes the event?

Usually:

The aggregate creates the event.

But the aggregate does not directly publish it.

Example:

Inside Order:

```java public void confirm(){

status = CONFIRMED;

addDomainEvent(
    new OrderConfirmed(id)
);

} ```

The aggregate records:

```
text "Something happened."
```

---

Then:

Application layer or infrastructure publishes it.

Flow:

```text Order Aggregate

    |
    ↓

Creates Domain Event

    |
    ↓

Application Service

    |
    ↓

Event Publisher

    |
    ↓

Message Broker ```

---

# 6. Who consumes the event?

Other parts of the system that are interested.

Example:

Event:

```
text OrderPlaced
```

Consumers:

Inventory Service:

```
text Reserve stock
```

Payment Service:

```
text Process payment
```

Notification Service:

```
text Send email
```

Analytics Service:

```
text Record purchase
```

Important:

The publisher does not know who consumes.

This creates loose coupling.

---

7. Should the aggregate publish directly to Kafka?

No.

The aggregate should not know about Kafka.

Bad:

```java class Order {

KafkaProducer producer;

void confirm(){

    producer.send(
      "OrderConfirmed"
    );

}

} ```

Why bad?

Now the domain depends on:

- Kafka
- Messaging technology
- Infrastructure

---

Better:

```text Order Aggregate

creates event

↓

Application Layer

publishes event

↓

Kafka ```

---

The domain says:

> "OrderConfirmed happened."

Infrastructure decides:

> "How should we communicate it?"

---

# 8. What's the difference between domain event and integration event?

Very important distinction.

---

### Domain Event

Used inside a bounded context.

Purpose:

> Notify parts of the same domain model.

Example:

Inside Order Context:

```
text OrderConfirmed
```

Consumed by:

- Order process
- Pricing logic
- Internal handlers

Integration Event

Used between bounded contexts or services.

Purpose:

> Communicate with external systems.

Example:

Order Service publishes:

```
text OrderPlacedIntegrationEvent
```

to:

- Inventory Service
- Shipping Service
- Notification Service

Comparison:

| Domain Event | Integration Event | | ---------- | ---------- | | Internal | External | | Same bounded context | Across boundaries | | Rich domain meaning | Stable communication contract | | Can change more freely | Requires compatibility |

Flow:

```text Order Aggregate

    ↓

Domain Event

    ↓

Application Layer

    ↓

Integration Event

    ↓

Other Services ```
```

---

9. Should domain events leave the bounded context?

Usually, no.

A bounded context owns its language.

Example:

Order Context:

```
text OrderConfirmed
```

may make sense internally.

But another context may need a different representation.

Example:

Shipping Context does not need:

```
text Order Aggregate details
```

It needs:

```
text ShipmentRequested
```

Therefore:

Translate internal events into external events.

Example:

```text OrderConfirmed

    ↓

OrderShipmentRequested ```

---

# 10. What information should an event contain?

An event should contain enough information for consumers to react.

Usually:

- Event type
- Aggregate ID
- Relevant business data
- Timestamp
- Event metadata

---

Example:

```
json { "eventType":"OrderPlaced", "orderId":"ORD-1001", "customerId":"CUS-500",
"items":[{ "productId":"P1", "quantity":2 }], "occurredAt":"2026-08-28T10:00:00" }
```

---

Avoid including the entire aggregate.

Bad:

```
json { "entireOrderObject": {} }
```

Why?

Creates tight coupling.

---

11. Event ID, aggregate ID, timestamp, version: which metadata is useful?

These are very useful.

---

1. Event ID

Example:

```
text eventId: 8a92-33ff
```

Purpose:
```

Detect duplicate events.

---

2. Aggregate ID

Example:

```
text orderId: ORD-1001
```

Purpose:

Know which entity changed.

---

3. Timestamp

Example:

```
text occurredAt: 2026-08-28
```

Purpose:

Ordering and auditing.

---

4. Version

Example:

```
text aggregateVersion: 5
```

Purpose:

Concurrency control.

Example:

Order changed:

```text Version 4

↓

OrderConfirmed

↓

Version 5 ```

---

Common event metadata:

```
json { "eventId":"", "eventType":"", "aggregateId":"", "version":"", "occurredAt":"" }
```

---

## 12. How are events persisted?

There are several approaches.

---

### 1. In-memory until transaction completes

Simple approach:

Aggregate stores events temporarily.

Example:

```
java order.addDomainEvent( new OrderPlaced() );
```

After saving:

Events are published.

```

2. Event Store

Used in Event Sourcing.

Events become the source of truth.

Example:

Instead of storing:

```
text Order Table
```

Store:

```text OrderCreated

PaymentReceived

OrderShipped ```

---

### 3. Outbox Table

Most common in microservices.

Database:

```text orders

outbox_events ```

The event is stored with business data.

---

## 13. What happens if database commit succeeds but event publishing fails?

This is a classic distributed systems problem.

Example:

Step 1:

Save order:

```
text Order Created
```

Database:

✅ Success

Step 2:

Publish event:

```
text OrderPlaced
```

Kafka:

❌ Failure

Now:

Database says:

```
text Order exists
```

But:

Other services don't know.

Problem:

```text System inconsistency
```

This is why we need reliable event publishing.

---

14. How does Transactional Outbox Pattern solve this?

Transactional Outbox solves the database + messaging consistency problem.

Instead of:

```text Save Database

then

Publish Message ```

we do:

```text Save Business Data

+

Save Event Record

in SAME transaction ```

---

Example:

Transaction:

```text BEGIN

Insert Order

Insert Outbox Event

COMMIT ```

Database:

```text Orders Table

+

Outbox Table ```

Both succeed together.

---

Then a separate process:

```text Outbox Publisher

    ↓

Kafka

    ↓

Consumers ```

---

Architecture:

```text Application Service

    |
    ↓

Database Transaction
```

| | Order Table Outbox Table | |

```

↓

Outbox Processor

↓

Kafka ```

---

15. How do consumers handle duplicate events?

Because distributed messaging often provides:

At-least-once delivery

Meaning:

The same event may arrive more than once.

Example:

Consumer receives:

```
text OrderPlaced
```

Processes it.

Then receives again:

```
text OrderPlaced
```

Solution:

Make consumers idempotent.

---

16. What is idempotency?

Idempotency means:

> Performing the same operation multiple times produces the same result as performing it once.

Example:

Payment:

First message:

```
text Charge $100
```

Result:

```
text Payment completed
```

Duplicate message:

```
text Charge $100 again
```

Should not create:

```
text Two payments
```

Implementation:

Store processed event IDs.

Example:

Database:

```text processed_events

event_id status ```

Before processing:
```

```
text Have I seen this event?
```

If yes:

Ignore.

---

17. What is event ordering?

Event ordering means events should be processed in the correct sequence.

Example:

Correct:

```text OrderCreated

↓

PaymentCompleted

↓

OrderShipped ```

Incorrect:

```text OrderShipped

↓

OrderCreated ```

---

Solutions:

## 1. Aggregate version

Example:

```
text Order Version 1 Order Version 2 Order Version 3
```

---

2. Message partitioning

Kafka can keep ordering within a partition.

Example:

All events for:

```
text Order ID = 1001
```

go to same partition.

---

18. What does at-least-once delivery mean for domain events?

At-least-once means:

> The system guarantees delivery, but duplicates may happen.

Example:

Event:

```
text PaymentCompleted
```

Consumer may receive:

```text PaymentCompleted

PaymentCompleted ```

---
```

Advantages:

You don't lose important messages.

Disadvantage:

Consumers must handle duplicates.

Other delivery models:

At-most-once

May lose messages.

Exactly-once

Very difficult in distributed systems.

Most real systems use:

```text At-least-once

+

Idempotent consumers ```

---

## 19. How do schema changes affect old consumers?

Events are contracts.

Changing them can break consumers.

Example:

Old event:

```
json { "customerName":"Rahim" }
```

New event:

```
json { "fullName":"Rahim Ahmed" }
```

Old consumers break.

Solutions:

1. Version events

Example:

```text OrderPlacedV1

OrderPlacedV2 ```

---

### 2. Add fields instead of removing

Good:

```
json { "name":"Rahim", "email":"test@test.com" }
```

Old consumers ignore new fields.

---

3. Maintain backward compatibility

Consumers should tolerate changes.

Complete Flow Example

Order placement:

Step 1

Customer places order.

Command:

```
text PlaceOrder
```

Step 2

Order Aggregate:

```text Creates:

OrderPlaced Event ```

---

## Step 3

Application Service saves:

```text Order

+

Outbox Event ```

---

## Step 4

Outbox publisher sends:

```
text OrderPlacedIntegrationEvent
```

Step 5

Consumers react:

Inventory:

```
text Reserve Stock
```

Payment:

```
text Charge Customer
```

Notification:

```
text Send Email
```

Core principle to remember:

> A domain event represents a business fact, not a technical message.

A good event:

```text ✓ Uses past tense

✓ Represents something meaningful

✓ Is created by the domain

✓ Does not know infrastructure

✓ Contains necessary information

✓ Supports reliable communication ```
```

The evolution path is:

```text Domain Event

↓

Integration Event

↓

Message Broker

↓

Distributed System ```

This is where Domain-Driven Design naturally leads into microservices, event-driven architecture, and distributed systems.

---

$$\text{Context Mapping}$$

---

# 14. Context Mapping

Context Mapping is where Domain-Driven Design moves from designing **one bounded context** to understanding how **multiple bounded contexts interact with each other**.

A simple way to remember:

> A context map describes the relationships, dependencies, and communication patterns between bounded contexts.

A bounded context tells us:

> "Where does a model apply?"

A context map tells us:

> "How do different models communicate with each other?"

---

# 1. What is context mapping?

**Context Mapping** is a DDD technique used to visualize and document the relationship between different bounded contexts in a system.

It shows:

- Which contexts exist.
- Which contexts depend on others.
- Who controls the relationship.
- How models are translated.
- Where coupling exists.

---

Example:

An e-commerce system:

```text id="ctx1" Customer Context

|
↓

Order Context

|
↓

Payment Context

|
↓

Shipping Context ```

A context map explains:

- Does Order follow Payment's model?
- Does Payment adapt to Order?
- Do both teams collaborate?
- Is there a translation layer?

---

Without context mapping:

Teams may accidentally create:

```text id="ctx2" Shared database

Shared models

Hidden dependencies ```

---

# 2. Why do bounded contexts need explicit relationships?

Because bounded contexts are not isolated islands.

In real systems:

```text id="ctx3" Order Context

needs information from

Customer Context ```

or:

```text id="ctx4" Shipping Context

needs

Order information ```

---

If relationships are not clear, problems appear:

## 1. Hidden coupling

Example:

Order team directly depends on Customer database.

```text id="ctx5" Order Service

    |
    ↓

Customer Database ```

Now Customer changes can break Order.

---

## 2. Conflicting models

Example:

Customer means:

Sales:

```
text id="ctx6" Potential buyer
```

Billing:

```
text id="ctx7" Person responsible for payment
```

Without clear boundaries, teams argue:

"Which Customer model is correct?"

Answer:

Both can be correct in their own context.

---

3. Uncontrolled dependencies

One team becomes dependent on another team's decisions.

Context mapping makes these relationships visible.

---

3. What information should a context map contain?

A context map usually contains:

---

1. Bounded Contexts

Example:

```text id="ctx8" Sales Context

Order Context

Payment Context

Shipping Context ```

---

### 2. Relationship between contexts

Example:

```text id="ctx9" Order Context

    uses

Payment Context ```

---

### 3. Direction of influence

Who controls the relationship?

Example:

```text id="ctx10" Payment Context

    ↓

Order Context ```

Payment is upstream.

Order is downstream.

---

### 4. Communication style

Example:

- API
- Events
- Messages
- Shared model
- Translation layer

---

### 5. Ownership

Who owns the model?

Example:
```

```text id="ctx11" Team A owns:

Customer Context

Team B owns:

Order Context ```

---

A context map is not just a technical diagram.

It is a **business and organizational map**.

---

# 4. What is an upstream context?

An **upstream context** is a bounded context that provides information or capabilities to another context.

The upstream context influences the downstream context.

Example:

```text id="ctx12"

Payment Context

    ↓

Order Context ```

Payment is upstream because Order depends on Payment information.

---

The upstream context has more power because:

- It controls the model.
- Changes can affect downstream systems.

---

Example:

Payment team changes:

```text id="ctx13" PaymentStatus

PENDING SUCCESS FAILED ```

Order team may need to adapt.

---

# 5. What is a downstream context?

A **downstream context** depends on another context.

It consumes information from the upstream context.

Example:

```text id="ctx14"

Customer Context

    ↓

Order Context ```

Order depends on customer information.

Order is downstream.

---

The downstream context must decide:

- Follow upstream model.
- Translate upstream model.
- Protect itself.

---

Example:

Customer Context says:

```text id="ctx15" CustomerStatus

ACTIVE BLOCKED ```

Order may translate:

```text id="ctx16" CanPlaceOrder

YES NO ```

---

# 6. What happens when one team's model influences another team's model?

This is a common organizational problem.

Example:

Payment Team owns:

```
text id="ctx17" Payment Model
```

Order Team uses:

```
text id="ctx18" Payment Status
```

If Order directly copies Payment's model:

```
text id="ctx19" PaymentStatus = SUCCESS
```

then Order becomes dependent.

Problems:

1. Change propagation

Payment changes:

```text id="ctx20" SUCCESS

becomes

COMPLETED ```

Order breaks.

---

## 2. Loss of autonomy

Order cannot evolve independently.

---

## 3. Model corruption

Payment concepts enter Order domain.

Example:

Order starts talking about:

```
text id="ctx21" Payment Gateway Authorization Code
```

even though Order does not own payments.

---

Solutions:

- Anti-Corruption Layer
- Published Language
- Separate models
```

7. What are the major context-map relationships?

DDD defines several common patterns.

---

1. Partnership

Two teams cooperate closely.

Relationship:

```text id="ctx22" Team A

⇄

Team B ```

Characteristics:

- Both teams succeed or fail together.
- Changes are coordinated.
- Strong collaboration.

Example:

Order Team and Inventory Team during early development.

---

Advantages:

✓ Good communication ✓ Fast alignment

Disadvantages:

✗ Requires close coordination

---

# 2. Shared Kernel

Two contexts share a small common model.

Example:

```text id="ctx23"

Sales Context

    |
    |

Shared Kernel

    |
    |

Billing Context ```

Shared code:

```text id="ctx24" CustomerId

Money

Address ```

---

Benefits:

- Avoid duplicate models.

Risks:

- Creates coupling.
- Changes require agreement.

---
```

Rule:

Shared Kernel should be:

```text id="ctx25" Small

Stable

Carefully managed ```

---

# 3. Customer/Supplier

One context provides services, another consumes them.

Like a business relationship.

Example:

```text id="ctx26"

Upstream:

Payment Context

Downstream:

Order Context ```

---

The supplier should consider customer needs.

The customer communicates requirements.

---

# 4. Conformist

The downstream context simply accepts the upstream model.

Example:

```text id="ctx27"

External Tax Service

↓

Billing Context ```

Billing uses the tax service model directly.

---

When used:

- Upstream system is powerful.
- Changing it is impossible.

Example:

Government tax API.

---

Problem:

Downstream loses control.

---

# 5. Anti-Corruption Layer (ACL)

One of the most important patterns.

An ACL protects one context from another context's model.

Example:
```

```text id="ctx28"
Legacy System
    ↓
Anti-Corruption Layer
    ↓
New Order System ```

---

The ACL translates:

Legacy:

```
text id="ctx29" Cust_Code
```

New system:

```
text id="ctx30" CustomerId
```

Benefits:

✓ Protects domain model ✓ Reduces coupling ✓ Allows independent evolution

---

6. Open Host Service

A context provides a well-designed public interface.

Example:

```text id="ctx31"
Customer Context
    |
    ↓
Customer API ```

Other systems consume the API.

---

The service says:

> "Here is the official way to communicate with me."

---

Example:

```
http GET /customers/{id}
```

---

7. Published Language

A shared, documented communication language.

Example:

Event contract:

```
json id="ctx32" { "event":"OrderPlaced", "orderId":"123" }
```

Consumers understand this format.

Used for:

- APIs
- Events
- Messages

Benefits:

- Clear communication.
- Less misunderstanding.

---

8. Separate Ways

Contexts have no meaningful relationship.

Example:

```text id="ctx33"
Reporting System

    x

Payment System
```

No integration needed.

Instead of forcing communication:

Keep them separate.

---

8. Which relationships create tight coupling?

Most coupled:

Shared Kernel

Because:

```text id="ctx34"
One model

shared by multiple contexts
```

A change affects everyone.

Conformist

Because:

```
text id="ctx35" Downstream follows upstream completely.
```

Partnership

Because:

```
text id="ctx36" Teams coordinate constantly.
```

High coupling:

```text id="ctx37"
Shared Kernel

    ↓

Conformist

    ↓

Partnership
```

---

9. Which relationships protect autonomy?

More autonomous:

Anti-Corruption Layer

Because:

text id="ctx38" Translation boundary exists.

Separate Ways

No dependency.

Open Host Service

Clear contract.

Better autonomy:

```text id="ctx39"
Separate Ways
    ↓
ACL
    ↓
Open Host Service
```

---

10. How do context maps relate to organization/team structures?

Context maps often reflect Conway's Law.

Conway's Law:

> Systems tend to mirror the communication structure of organizations.

Example:

Organization:

```text id="ctx40"
Sales Team
Payment Team
Shipping Team
```

Architecture:

```text id="ctx41"
Sales Context
Payment Context
Shipping Context
```

If two teams:

- Have separate goals.
- Own separate decisions.
- Communicate through contracts.

They likely need separate contexts.

Bad alignment:

```text id="ctx42"
```

5 Teams

↓

One Giant Shared Model ```

Creates conflicts.

Good alignment:

```text id="ctx43"
One Team

↓

One Context

↓

One Ownership Boundary ```

---

# 11. How do context maps translate into microservice dependencies?

Context maps help define:

- Service boundaries.
- API relationships.
- Event communication.
- Ownership.

Example:

Context map:

```text id="ctx44"
Order Context

    |
    |
    ↓

Payment Context ```

Microservices:

```text id="ctx45"
Order Service

    |
    |

REST/Event

    ↓

Payment Service ```

---

Example with ACL:

Context:

```text id="ctx46"
Legacy Billing

    ↓

ACL

    ↓

New Payment Context ```

Microservices:

```text id="ctx47"
Legacy Billing Service
    ↓
Translation Service
    ↓
Payment Service ```

---

Context mapping prevents:

`text id="ctx48" Microservice spaghetti`

where every service directly depends on every other service.

---

# Complete Example: E-commerce Context Map

```text id="ctx49"
Customer Context | | ↓
Order Context | | ↓
Payment Context | | ↓
Shipping Context ```

Possible relationships:

```text id="ctx50"
Customer → Order
Customer/Supplier
Order → Payment
Open Host Service
Legacy Payment → Order
Anti-Corruption Layer ```

---

# Core principle to remember:

> A bounded context defines your model. A context map defines how your models cooperate.

A healthy context map:

```text id="ctx51"
✓ Clear ownership
✓ Explicit relationships
✓ Controlled dependencies
✓ Translation where needed
✓ Team autonomy
✓ Stable communication contracts ```

The key question when designing context relationships:

> "How can these teams collaborate without one team's model corrupting another team's model?"

---

$$\text{Anti-Corruption Layer — ACL}$$

# 15. Anti-Corruption Layer (ACL)

Anti-Corruption Layer is one of the most important DDD patterns for protecting your **bounded context from external models**.

A simple way to remember:

> An Anti-Corruption Layer acts as a translator and protective barrier between your domain model and an external system's model.

It prevents foreign concepts from entering and damaging your domain language.

---

# 1. What is an Anti-Corruption Layer?

An **Anti-Corruption Layer (ACL)** is a layer that translates communication between two bounded contexts while protecting your own domain model.

It sits between:

```text id="acl1"
External System
    ↓
Anti-Corruption Layer
    ↓
Your Domain Model
```

Example:

External banking system:

```
json { "cust_no": 8171, "acct_stat": "A" }
```

Your domain:

```java id="acl2"
CustomerId customerId;

AccountStatus.ACTIVE;
```

The ACL converts:

```text id="acl3"
cust_no
    ↓
CustomerId
```

and:

```text id="acl4"
acct_stat = "A"
    ↓
AccountStatus.ACTIVE
```

Your domain never knows about:

```text id="acl5"
cust_no

acct_stat

Legacy naming
```

---

2. Why is it called anti-corruption?

Because external models can **corrupt your domain model**.

Imagine a company integrates with a legacy banking system.

Legacy system uses:

```text id="acl6" CUST_NO

ACCT_STAT

TRX_CD ```

A developer directly uses these names:

```java class Customer {

String cust_no;

String acct_stat;

} ```

Now your clean domain becomes polluted by external terminology.

The domain starts speaking the language of the external system.

This is corruption.

---

The ACL prevents:

```text id="acl7"

External Language

    ↓

Your Domain

```

Instead:

```text id="acl8"

External Language

    ↓

Translator

    ↓

Your Domain Language ```

---

# 3. What exactly is being protected?

The ACL protects your:

## 1. Domain Model

Example:

Your model:

`java CustomerId AccountStatus`

should not become:

`java cust_no acct_stat`

---

## 2. Ubiquitous Language

Your team has its own business language.

Example:

Your domain:

```text id="acl9" Customer

Account

Active ```

External system:

```text id="acl10" Party

Acct

A ```

The ACL protects your terminology.

---

### 3. Business Rules

External systems may have different rules.

Example:

External system:

```
text id="acl11" A = Active
```

Your domain:

```text id="acl12" ACTIVE means:

Customer can perform transactions. ```

The ACL translates meaning, not only fields.

---

## 4. When should an ACL be introduced?

Use an ACL when the external system has a model that should not enter your domain.

Common situations:

---

### 1. Legacy system integration

Example:

Old ERP system:

```
text id="acl13" CUSTOMER_MASTER ORDER_STATUS_CD
```

Your new system:

```
text id="acl14" Customer OrderStatus
```

Use ACL.

---

2. Third-party systems

Example:

Payment provider:

```
json { "txn_code":"00", "resp":"OK" }
```

Your domain:

```
java PaymentResult.SUCCESS
```

ACL translates.

---

3. Different bounded contexts

Example:

Sales Context:

text id="acl15" Lead

Billing Context:

text id="acl16" Account

They need translation.

---

4. When you do not control the external model

If another team changes their model whenever they want, ACL protects you.

---

5. How does an ACL protect the domain model?

It creates a translation boundary.

Without ACL:

```text id="acl17"
External API DTO
    ↓
Domain Entity
```

Problem:

External concepts leak inside.

With ACL:

```text id="acl18"
External DTO
    ↓
Translator
    ↓
Domain Object
```

Example:

External:

json { "cust_no":8171, "acct_stat":"A" }

ACL:

```java Customer convert(CustomerResponse response){
return new Customer( new CustomerId(response.cust_no), AccountStatus.ACTIVE );
} ```

Domain receives:

```java CustomerId(8171)
AccountStatus.ACTIVE ```

---

The domain remains clean.

---

# 6. What is the difference between ACL and API Gateway?

They solve different problems.

---

## API Gateway

Concern:

> Routing and managing external traffic.

Responsibilities:

- Authentication
- Rate limiting
- Routing
- Load balancing
- Request aggregation

Example:

```text id="acl19"
Mobile App
   ↓
API Gateway
   ↓
Microservices
```

Anti-Corruption Layer

Concern:

> Protecting domain meaning.

Responsibilities:

- Translation
- Model adaptation
- Terminology conversion
- Business meaning protection

Example:

```text id="acl20"
Legacy System
   ↓
ACL
   ↓
Order Domain
```

Comparison:

API Gateway	ACL
Infrastructure pattern	DDD pattern
Handles traffic	Handles meaning
Technical concern	Domain protection
Outside entry point	Translation boundary

---

7. What is the difference between ACL and Adapter?

They are related but different.

Adapter

A general software design pattern.

Purpose:

> Make two incompatible interfaces work together.

Example:

```
java id="acl21" OldPrinterAdapter
```

Converts:

```text Old Printer API

    ↓

New Printer Interface ```

---

## ACL

A DDD pattern.

Purpose:

> Protect one domain model from another domain model.

It often uses adapters internally.

---

Relationship:

```text id="acl22"

ACL

contains

Adapters + Translators + Mappers

```

Example:

Adapter:

```
text id="acl23" Convert HTTP response format
```

ACL:

```
text id="acl24" Convert Banking concepts into our Customer domain
```

---

8. What is the relationship between ACL and Hexagonal Architecture?

They work very well together.

Hexagonal Architecture says:

> The domain should be protected from external systems through ports and adapters.

Architecture:

```text id="acl25"

External System

    ↓

Adapter / ACL

    ↓
```
```

Application Port

↓

Domain Model

```

---

Example:

Your domain defines:

```java interface CustomerProvider {

```
Customer find(CustomerId id);
```

} ```

External implementation:

`java LegacyCustomerAdapter`

The adapter contains ACL logic.

The domain only knows:

`text id="acl26" CustomerProvider`

Not:

```text id="acl27" REST API

SOAP

Legacy Database ```

---

# 9. Should external DTOs enter the domain directly?

No.

This is one of the biggest mistakes.

Bad:

```java class Customer {

```
LegacyCustomerDTO dto;
```

} ```

Now the domain depends on external representation.

Problem:

External change:

`json cust_no`

becomes:

`java customer.cust_no`

inside your domain.

Better:

```text id="acl28"

External DTO

↓

ACL Mapper
```

↓

Domain Object

```

---

Example:

External DTO:

```java class LegacyCustomerDTO {

String cust_no;

String acct_stat;

} ```

Domain:

```java class Customer {

CustomerId id;

AccountStatus status;

} ```

---

# 10. Where should translation happen?

Translation should happen at the boundary.

Not inside the domain.

Not inside controllers.

---

Good:

```text id="acl29"

External API

↓

ACL Layer

↓

Application Layer

↓

Domain

```

Bad:

```text id="acl30"

Controller

does translation

+ business logic

```

or:

```text id="acl31"

Domain Entity

understands external format

```

The domain should receive clean concepts.

---

11. How do we map external terminology into our ubiquitous language?

By translating concepts, not just names.

Example:

External:

```
text id="acl32" acct_stat = "A"
```

Does not mean anything in your domain.

ACL understands:

```
text id="acl33" "A" means account is active
```

Converts:

```
java AccountStatus.ACTIVE
```

Example:

External:

```
text id="acl34" CUST_NO
```

Your domain:

```
java CustomerId
```

The ACL performs:

```text id="acl35"
External Language
    ↓
Business Meaning
    ↓
Domain Language
```

---

12. How does an ACL help when integrating legacy systems?

Legacy systems often have:

- Old naming.
- Old architecture.
- Strange data formats.
- Different business assumptions.

Example:

Legacy:

```
json { "C_NO":8171, "STAT":"1" }
```

Your domain:

```java Customer {

CustomerId id;

Status status;

} ```

ACL:

```text id="acl36"

C_NO

↓

CustomerId

STAT=1

↓

ACTIVE

```

Benefits:

- Legacy system stays unchanged.
- New domain stays clean.
- Migration becomes easier.

---

13. How does an ACL help when integrating third-party systems?

Third-party APIs change.

Example:

Payment provider:

Version 1:

```
json { "status":"OK" }
```

Version 2:

```
json { "result":"SUCCESS" }
```

Without ACL:

Your domain breaks.

With ACL:

Only ACL changes:

```text id="acl37"

Payment API v1/v2

    ↓

Payment ACL

    ↓

PaymentResult.SUCCESS

```

The domain remains stable.

---

14. What are the maintenance costs of an ACL?

ACL provides protection, but it has costs.

---

1. More code

You need:

- DTOs
- Mappers
- Translators
- Adapters

Example:

```text id="acl38"
ExternalCustomerDTO
    ↓
CustomerMapper
    ↓
Customer
```

---

2. Mapping maintenance

When external API changes:

ACL must change.

---

3. Additional complexity

For simple integrations, ACL may be unnecessary.

Example:

Small internal service:

```text id="acl39"
Service A
    ↓
Service B
```

A simple API client may be enough.

Use ACL when:

- External model is complex.
- External system changes frequently.
- Domain independence is important.

Complete Example

Without ACL

```text id="acl40"
Legacy Banking System
    ↓
Customer Domain
```

Customer {

String cust_no;

String acct_stat;

}

```

Problem:

Domain speaks legacy language.

---

### With ACL

```text id="acl41"

Legacy Banking System

↓

Legacy Customer Adapter

↓

ACL Translator

↓

Customer Domain

Customer {

CustomerId id;

AccountStatus status;

}

```

Core principle to remember:

An Anti-Corruption Layer protects your domain from learning the language of another system.

A good ACL:

```text id="acl42"

✓ Translates models

✓ Protects ubiquitous language

✓ Protects business rules

✓ Isolates legacy systems

✓ Keeps domain clean

✓ Allows independent evolution ```

The key question:

"If this external system changes tomorrow, will my domain model still make sense?"

If the answer is yes, your ACL is doing its job.

---

$$\text{CQRS}$$

---

# 16. CQRS (Command Query Responsibility Segregation)

CQRS is where we move from **domain modeling inside one application** into **architectural design for complex systems**.

It is a pattern that separates two different responsibilities:

- Changing the system state.
- Reading information from the system.

A simple definition:

> CQRS separates the model used for writing data from the model used for reading data.

---

Traditional systems assume:

```text id="cqrs1"
Create Data

Update Data

Read Data

↓

Same Model

↓

Same Database
```

CQRS says:

```text id="cqrs2"
        Command

Client ----------→ Write Model | | ↓ Write Database

Client ----------→ Query Model

        Query

                    ↓

              Read Database
```

---

1. What is CQRS?

CQRS stands for:

Command Query Responsibility Segregation

It means:

- Commands modify state.
- Queries retrieve information.

The write side and read side have different responsibilities.

Example:

E-commerce system.

Traditional CRUD:

```text id="cqrs3"
Order Table

|
|

Create Order Update Order Read Order
```

Same model handles everything.

CQRS:

Write side:

```text id="cqrs4"

Order Command Model

Responsibilities:

- Place order
- Cancel order
- Confirm payment

```

Read side:

```text id="cqrs5"

Order Read Model

Responsibilities:

- Show order history
- Display dashboard
- Search orders

```

---

2. What does Command Query Responsibility Segregation actually mean?

Let's break the words.

Command

A request to change something.

Example:

```text id="cqrs6"

PlaceOrder

CancelOrder

ApproveLoan

RegisterCustomer

```

A command says:

> "Please perform this action."

Query

A request to get information.

Example:

```text id="cqrs7"

GetCustomerDetails

SearchOrders

GetDashboard
```

```

A query says:

> "Give me information."

---

### Responsibility Segregation

Means:

These two responsibilities should not necessarily share the same model.

---

Traditional:

```text id="cqrs8"
One model
does everything
```

CQRS:

```text id="cqrs9"
Write model
handles business rules
Read model
handles data retrieval
```

---

3. What is a command?

A **command** represents an intention to change the system.

Examples:

```text id="cqrs10"
CreateOrder
ChangeAddress
ApprovePayment
CancelSubscription
```

Characteristics:

- Changes state.
- Contains intent.
- Usually does not return data.

Example:

```java id="cqrs11"
PlaceOrderCommand {
CustomerId customerId;
List items;
}
```

The command says:

> "Place this order."

It does not say:
```

"Update order table."

Good command:

```
text id="cqrs12" ApproveLoan
```

Bad command:

```
text id="cqrs13" UpdateLoanStatus
```

Why?

Because:

```text id="cqrs14" ApproveLoan
```

has business meaning. ```

---

4. What is a query?

A query retrieves information without changing state.

Examples:

```text id="cqrs15"

GetOrderDetails

SearchCustomers

GetSalesReport

```

Characteristics:

- No side effects.
- Does not modify data.
- Optimized for reading.

Example:

```java id="cqrs16" GetCustomerOrdersQuery {

CustomerId id;

} ```

Returns:

```
json id="cqrs17" { "name":"Rahim", "orders":[...] }
```

---

# 5. Why shouldn't commands return complex read models?

Because commands and queries have different responsibilities.

Example:

Bad:

```
java id="cqrs18" Order order = placeOrder();
```

Now the command is doing:

1. Changing state.
2. Building a read response.

These are different jobs.

Problem:
```

After placing an order, someone asks:

"Return customer information, product details, shipping status, payment history."

Now your command depends on many models.

Better:

Command:

```text id="cqrs19" PlaceOrder

returns:

OrderId ```

Example:

```
json id="cqrs20" { "orderId":"ORD-1001" }
```

Then query:

```
text id="cqrs21" GetOrderDetails
```

returns:

```
json id="cqrs22" { "orderId":"ORD-1001", "customer":"Rahim", "items":[...],
"payment":"Completed" }
```

---

6. Does CQRS require two databases?

No.

This is a common misunderstanding.

CQRS can exist at different levels.

Level 1: Logical CQRS

Same database.

Different models.

Example:

```text id="cqrs23"

Command Model

    |
    |

Same Database

    |
    |

Query Model

```

Example:

Spring application:

```text id="cqrs24"

OrderService

OrderCommandHandler

OrderQueryService

```
```

Level 2: Physical CQRS

Separate storage.

Example:

```text id="cqrs25"
Write Database
(PostgreSQL)
    ↓
Events
    ↓
Read Database
(ElasticSearch)
```

Two databases are optional.

CQRS means separation of responsibility, not necessarily storage.

---

7. Does CQRS require event sourcing?

No.

They are separate concepts.

Many people confuse them.

CQRS:

Separates:

```text id="cqrs26"
Write
and
Read
```

Event Sourcing:

Stores:

```text id="cqrs27"
Events
instead of current state
```

Example:

Instead of:

```text id="cqrs28" Order
Status = Paid
```

Store:

```text id="cqrs29"
OrderCreated
PaymentReceived
OrderShipped
```

They can be combined:

```text id="cqrs30"
CQRS
+
Event Sourcing
```

But:

CQRS can exist without Event Sourcing.

---

8. Can CQRS exist inside one application?

Yes.

CQRS does not require microservices.

Example:

Single Spring Boot application:

```text id="cqrs31"
OrderController
    ↓
Command Handler
    ↓
Domain Model

Query Controller
    ↓
Query Service
```

Same deployment.

Example:

```java id="cqrs32" PlaceOrderHandler
GetOrderHandler ```

Different responsibilities.

---
```

9. What is logical CQRS versus physical CQRS?

Logical CQRS

Separation in code.

Example:

```text id="cqrs33"

OrderCommandService

OrderQueryService

```

Same database.

Physical CQRS

Separate infrastructure.

Example:

```text id="cqrs34"

Write DB

↓

Event Stream

↓

Read DB

```

Comparison:

| Logical CQRS | Physical CQRS | | ---------- | ---------- | | Code separation | Infrastructure separation | | Simple | More complex | | Same database possible | Separate databases | | Easier to adopt | Better scalability |

---

10. Why might the write model differ from the read model?

Because writing and reading have different needs.

Write model cares about:

- Business rules.
- Consistency.
- Validation.

Example:

```text id="cqrs35"

Order Aggregate

```

Read model cares about:

- Speed.
- Searching.
- Reporting.
- Display.

Example:

Dashboard:

```text id="cqrs36"
Customer Name
Order Count
Revenue
Last Purchase
```

A single model struggles.

Example:

Database designed for transactions:

```text id="cqrs37"
Normalized tables
Many joins
```

Not ideal for dashboards.

Read model can be optimized:

```text id="cqrs38"
Denormalized table
Fast queries
```

---

11. How are read models generated?

Usually through events.

Flow:

```text id="cqrs39"
Command
↓
Write Model
↓
Domain Event
↓
Event Handler
↓
Read Model Update
```

Example:

Command:

```
text id="cqrs40" PlaceOrder
```

Write side:

```
text id="cqrs41" OrderCreated
```

Event handler:

```
text id="cqrs42" Update Order View Table
```

Read database:

```text id="cqrs43"
OrderSummary
customerName
total
status
```

---

12. Why is CQRS useful in complex domains?

CQRS helps when:

1. Business rules are complicated

Example:

Banking:

```text id="cqrs44"
Loan approval
Risk calculation
Credit checking
```

---

2. Read and write needs are very different

Example:

Social media:

Writes:

```
text id="cqrs45" Create Post Like Post Comment
```

Reads:

```text id="cqrs46" News Feed
Recommendations
Trending ```

---

## 3. Different scaling requirements exist

Example:

Online shopping:

Writes:

```
text id="cqrs47" 10,000 orders/hour
```

Reads:
```

```
text id="cqrs48" Millions of product views/hour
```

Separate scaling helps.

---

13. What are its scalability advantages?

CQRS allows independent scaling.

Example:

Read traffic:

```text id="cqrs49"
100,000 users viewing dashboard
```

You scale:

```text id="cqrs50"
Read servers
```

without scaling:

```text id="cqrs51"
Write servers
```

Benefits:

1. Independent scaling

Read and write workloads scale separately.

---

2. Optimized storage

Write database:

```text id="cqrs52"
Normalized
Consistent
```

Read database:

```text id="cqrs53"
Denormalized
Fast
```

---

3. Better performance

Queries avoid complex joins.

---

14. What consistency problems does it introduce?

CQRS often introduces eventual consistency.

Example:

User places order.

Immediately:

Write database:

```text id="cqrs54"
Order Created
```

Read database:

```text id="cqrs55"
Not updated yet
```

User refreshes:

```text id="cqrs56"
Order not visible
```

A few seconds later:

```text id="cqrs57"
Order appears
```

This delay is the cost of separation.

---

15. What does eventual consistency look like from the user's perspective?

Example:

Bank transfer.

User sends money.

Write side:

```text id="cqrs58"
Transfer completed
```

Read side:

```text id="cqrs59"
Balance still old
```

After synchronization:

```text id="cqrs60"
Balance updated
```

User experience:

- Temporary delay.
- Data may appear slightly behind.
- Eventually becomes correct.

---

16. How do we handle stale reads?

Several approaches:

---

1. Inform users

Example:

"Your order is being processed."

---

2. Read from write model after important actions

Example:

Immediately after checkout:

```text id="cqrs61"
Show order from write database
```

---

3. Include versions

Example:

```text id="cqrs62"
Order Version 5
```

Client knows data freshness.

---

4. Refresh asynchronously

Example:

UI receives:

```text id="cqrs63"
Order updated
```

refreshes automatically.

---

17. When is CQRS unnecessary complexity?

CQRS is not for every system.

Avoid it when:

Simple CRUD applications

Example:

Employee management:

```text id="cqrs64"
```

Create Employee

Update Employee

Delete Employee

Search Employee

```

Normal CRUD is enough.

---

### Small domains

If:

- Few business rules.
- Low traffic.
- Simple reports.

CQRS adds unnecessary complexity.

---

Costs:

- More code.
- More infrastructure.
- Event handling.
- Monitoring complexity.
- Debugging difficulty.

---

## 18. How would you implement CQRS with Spring Boot?

Typical structure:

```text id="cqrs65"

controller

↓

command package

↓

Command Handler

↓

Domain Model

↓

Repository

---

query controller

↓

Query Service

↓

Read Repository

```

Example:

Command:

```java id="cqrs66" PlaceOrderCommand
```

Handler:
```

```java id="cqrs67" PlaceOrderHandler {

handle(command){

Order order = Order.create();

repository.save(order);

}

} ```

---

Query:

```java id="cqrs68" OrderQueryService {

findOrderView(id){

return readRepository.find(id);

}

} ```

---

# 19. How would Kafka fit?

Kafka is commonly used for communication between write and read sides.

Flow:

```text id="cqrs69"

Command

↓

Write Model

↓

Domain Event

↓

Kafka

↓

Read Model Consumer

↓

Read Database

```

Example:

Event:

```
json id="cqrs70" { "type":"OrderPlaced", "orderId":"1001" }
```

Consumer:

```text id="cqrs71"

Update Order View Table

```

Kafka provides:

- Event streaming.

- Decoupling.
- Scalability.

---

20. How would Redis/Elasticsearch/read replicas fit?

They are commonly used for read models.

Redis

Good for:

- Fast lookup.
- Caching.
- Frequently accessed data.

Example:

```text id="cqrs72"
User Profile Cache
```

Elasticsearch

Good for:

- Search.
- Filtering.
- Full-text queries.

Example:

```text id="cqrs73"
Product Search

Order Search
```

Read replicas

Database copies used for queries.

Example:

```text id="cqrs74"
Primary Database
      ↓
Read Replicas
```

Traditional CRUD vs CQRS

Traditional CRUD

```text id="cqrs75"
Client
↓
One Model
↓
```

One Database

```

Characteristics:

- Simple.
- Easy.
- Good for normal applications.

---

## CQRS

```text id="cqrs76"
          Command
Client ----------------→ Write Model
                              |
                              |
                              ↓
                      Write Database
Client ----------------→ Query Model
          Query
                              |
                              ↓
                      Read Database
```

Characteristics:

- Separate responsibilities.
- Different optimization.
- Better for complex systems.

Complete Example: Order System

Command Side

User:

```text id="cqrs77" Place Order
```

Handler:

```text id="cqrs78" Order Aggregate
```

Database:

```text id="cqrs79" Orders
```

Event:

```text id="cqrs80" OrderPlaced
```

Query Side

Event consumer:

```text id="cqrs81" OrderPlaced
    ↓
```

Create OrderView ```

Read database:

```text id="cqrs82" OrderView
```
```

Customer Name

Items

Payment Status

Delivery Status ```

Core principle to remember:

> CQRS separates the model that protects business rules from the model that answers questions quickly.

A healthy CQRS design:

```text id="cqrs83"
Commands
   ↓
Domain Model
   ↓
Events
   ↓
Read Models
   ↓
Queries
```

The key question:

> "Do my read needs and write needs have fundamentally different requirements?"

If yes, CQRS may be valuable.

If not, simple CRUD is often the better choice.

$$\text{Event Sourching}$$

---

17. Event Sourcing

Event Sourcing is one of the most advanced concepts in Domain-Driven Design. It is not a replacement for normal database persistence. It is a different way of thinking about **how we store the truth of a system**.

A simple definition:

> Event Sourcing stores the sequence of business events that changed an entity instead of storing only its current state.

Traditional systems store:

```
text Current State
```

Event Sourcing stores:

```
text History of Changes
```

Example:

Traditional Order table:

```text Order
ID: 1001 Status: SHIPPED Total: $500 ```
```

You only know the current situation.

Event Sourcing:

```text OrderCreated

↓

ItemAdded

↓

PaymentCompleted

↓

OrderShipped ```

The current state is calculated from history.

---

# 1. What is event sourcing?

Event Sourcing is a persistence pattern where application state is represented as a sequence of immutable events.

Instead of:

```text id="es1" Database

Order Table

---

ID Status Total ```

We store:

```text id="es2" Event Store

---

OrderCreated ItemAdded PaymentCompleted OrderShipped ```

---

The database becomes a historical record of everything that happened.

---

Example:

Bank Account.

Traditional:

```
text Account Balance = 5000
```

Event Sourcing:

```text AccountOpened

Deposit +10000

Withdraw -3000

Withdraw -2000 ```

Current balance:

```
10000 - 3000 - 2000 = 5000
```

---

2. How is it different from normal state persistence?

Traditional persistence:

Store the current truth.

Example:

```text Customer

Name: Rahim

Status: ACTIVE ```

If the name changes:

Before:

```
text Name = Rahim
```

After:

```
text Name = Rahim Ahmed
```

The old value is gone.

Event Sourcing:

Store the change:

```text CustomerRegistered

NameChanged ```

History:

```
text CustomerRegistered NameChanged
```

The current state is derived.

Comparison:

| Traditional Persistence | Event Sourcing | | ---------- | ---------- | | Stores current state | Stores events | | Updates records | Appends events | | Old data disappears | History remains | | Easy CRUD | More complex | | Simple reporting | Strong auditability |

---

3. Instead of storing current state, what exactly do we store?

We store **events**.

Events represent facts.

Example:

Instead of:

```
json { "status":"PAID" }
```

Store:

```
json { "type":"PaymentCompleted", "amount":500, "time":"2026-08-28" }
```

Example Order history:

```text OrderCreated

ProductAdded

DiscountApplied

PaymentCompleted

OrderShipped ```

---

Each event describes:

- What happened.
- When it happened.
```

- Which aggregate changed.
- Relevant information.

---

4. How is aggregate state reconstructed?

The current state is rebuilt by replaying events.

Example:

Event stream:

```text id="es3"
OrderCreated
↓
ItemAdded ($100)
↓
ItemAdded ($50)
↓
DiscountApplied ($20)
```

Starting state:

```
text Order = Empty
```

Apply events:

```
OrderCreated | ↓ Order exists
ItemAdded | ↓ Total = 100
ItemAdded | ↓ Total = 150
DiscountApplied | ↓ Total = 130
```

Final state:

```
text Order Total = 130
```

The aggregate rebuilds itself:

```java id="es4"
for(Event event : events){
```

```
apply(event);
```

```
}
```

---

5. What is an event stream?

An event stream is the ordered sequence of events belonging to one aggregate.

Example:

Order ID:

```
text ORD-1001
```

Events:

```text id="es5"
Version 1: OrderCreated
Version 2: ItemAdded
Version 3: PaymentCompleted
```

Version 4: OrderShipped ```

This is the event stream of that order.

Every aggregate usually has its own stream.

Example:

```text Order-1001 Stream

Customer-500 Stream

Account-900 Stream ```

---

# 6. What is aggregate versioning?

Aggregate versioning tracks the position of an event inside an aggregate's history.

Example:

```text Order ORD-1001

Version 1: OrderCreated

Version 2: ItemAdded

Version 3: PaymentCompleted ```

The version tells:

> "How many changes have happened to this aggregate?"

---

Why useful?

For:

- Concurrency control.
- Ordering.
- Detecting conflicts.

---

# 7. What is optimistic concurrency?

Optimistic concurrency assumes conflicts are rare and checks versions before saving.

Example:

Order version:

```
text Current Version = 5
```

User A loads:

```
Version 5
```

User B loads:

```
Version 5
```

User A updates:

```
Version 6
```

User B tries:

```
Save Version 6
```

But database says:

```
Expected Version 5 Current Version 6
```

Conflict.

This prevents overwriting someone else's changes.

Example:

```java id="es6"

save(events, expectedVersion=5) ```

If version changed:

Reject.

---

# 8. What is event replay?

Event replay means rebuilding state by processing historical events again.

Example:

Events:

```text CustomerRegistered

PurchasedProduct

EarnedPoints ```

Replay them:

```text Apply CustomerRegistered

Apply Purchase

Apply Points ```

Result:

Current customer state.

---

Uses:

## 1. Rebuilding databases

Example:

New read model:

```
text CustomerDashboard
```

Replay old events.

---

## 2. Debugging

See exactly what happened.

---

## 3. New features

Create new projections from old history.

---

# 9. What is a snapshot?

A snapshot is a saved copy of aggregate state at a specific point.

Instead of replaying:

```
text 10 million events
```
```

we store:

```
text Snapshot at Version 900000
```

Example:

Without snapshot:

```text Event 1

Event 2

Event 3

...

Event 1,000,000 ```

Need to replay everything.

---

With snapshot:

```text Snapshot Version 900000

+

Events 900001 → 1000000 ```

Much faster.

---

# 10. Why are snapshots useful?

Because event streams can become huge.

Example:

Bank account:

```text 10 years

Millions of transactions ```

Rebuilding balance from every transaction is expensive.

Snapshot:

```
text Balance after 9 years
```

Then replay only recent events.

---

Benefits:

- Faster loading.
- Better performance.
- Reduced computation.

---

# 11. Does event sourcing require CQRS?

No.

They are separate patterns.

---

Event Sourcing:

Focus:

```
text How data is stored
```

CQRS:
```

Focus:

```
text How reads and writes are separated
```

They work well together:

```text Event Store

  ↓

Events

  ↓

Read Models ```

But one does not require the other.

---

# 12. Does CQRS require event sourcing?

No.

CQRS can use normal databases.

Example:

Write:

```
text PostgreSQL
```

Read:

```
text ElasticSearch
```

No event store needed.

Many systems use:

```text CQRS

+

Normal Database ```

---

# 13. What is the difference between domain events and event-sourced events?

Very important.

---

## Domain Event

Represents an important business occurrence.

Example:

```
text OrderPlaced
```

Purpose:

Communication.

Event-Sourced Event

Represents a state-changing event stored permanently.

Example:

```
text OrderItemAdded PaymentReceived
```

Purpose:

Rebuild aggregate state.

Comparison:

| Domain Event | Event Sourced Event | | ---------- | ---------- | | For communication | For persistence | | May not be stored | Always stored | | Can be temporary | Immutable history | | Used by other systems | Used to rebuild state |

An event-sourced event is usually also a domain event, but not every domain event becomes stored.

---

14. Can stored events ever be modified?

Normally:

No.

Events are immutable.

Meaning:

Once written:

```
text PaymentCompleted
```

cannot become:

```
text PaymentCancelled
```

Why?

Because history must remain trustworthy.

Instead:

Create a new event:

```text PaymentCompleted

↓

PaymentRefunded ```

---

History:

```text Payment happened.

Then refund happened. ```

---

# 15. How do we handle incorrect historical events?

This is difficult.

Because events should not be changed.

Options:

---

## 1. Add correction events

Example:

Wrong:

```
text PriceUpdated 100
```

Add:

```
text PriceCorrectionApplied -20
```

---

2. Event migration

Transform old events into new format.

---

3. Rebuild projection logic

Sometimes the event is correct but interpretation changed.

Avoid modifying history.

---

16. How do we evolve event schemas?

Events are long-lived contracts.

Old events may be read years later.

Problems:

Old:

```
json { "name":"Rahim" }
```

New:

```
json { "fullName":"Rahim Ahmed" }
```

Solutions:

1. Version events

Example:

```text CustomerRegisteredV1

CustomerRegisteredV2 ```
```

---

2. Add fields safely

Good:

```
json { "name":"Rahim", "email":"x@test.com" }
```

Old consumers ignore email.

---

3. Upcasting

Convert old events into new format during reading.

---

17. What is event upcasting?

Event upcasting means transforming old event versions into the newest format.

Example:

Old event:

```json CustomerCreatedV1

{ "name":"Rahim" } ```
```

New format:

```json CustomerCreatedV2
```

{ "firstName":"Rahim", "lastName":"" } ```

When reading:

System converts:

```text V1

↓

V2 ```

before processing.

---

This allows history to remain unchanged.

---

# 18. How do we delete sensitive data under an immutable event history?

This is a major challenge.

Example:

GDPR:

"Delete customer information."

But events contain:

```text CustomerRegistered

Name

Email

Address ```

Events cannot be changed.

---

Solutions:

## 1. Store references instead of sensitive data

Example:

Instead of:

```
json { "email":"abc@test.com" }
```

Store:

```
json { "customerId":"123" }
```

---

2. Encrypt sensitive data

Destroy encryption key when deletion is required.

---

3. Separate personal data storage

Events contain:

```
text CustomerId
```

Personal data stored separately.

---

19. How does event sourcing support auditability?

Excellent.

Because every change exists.

Example:

Bank account:

History:

```text AccountOpened

DepositMade

WithdrawalMade

TransferCompleted ```

You know:

- Who changed it.
- When.
- Why.

---

Useful for:

- Banking.
- Healthcare.
- Insurance.
- Government systems.

---

## 20. How does it support temporal queries?

Temporal query means:

> "What was the state at a previous time?"

Example:

Question:

"What was this account balance on January 1st?"

Replay events until that date.

---

Traditional database:

Only current state.

Event sourcing:

Entire timeline.

---

## 21. What are its debugging benefits?

Huge advantage.

You can answer:

"What happened?"

Example:

Customer says:

"My order disappeared."

Replay:

```text OrderCreated

PaymentCompleted

OrderCancelled ```

You know exactly what happened.

---

Traditional system:

Only:

```
text Status = CANCELLED
```

History is gone.

---

22. What are its operational disadvantages?

Event sourcing adds complexity.

Problems:

1. Storage growth

Events grow forever.

---

2. More infrastructure

Need:

- Event store.
- Projection system.
- Monitoring.

---

3. Harder debugging

Not every developer understands event streams.

---

4. More complicated deployments

Schema changes become difficult.

---

23. How does event sourcing increase architectural complexity?

Normal CRUD:

```text Request

↓

Database Update ```

Simple.

---

Event sourcing:

```text Command

↓

Aggregate

↓

Create Event

↓

Store Event

↓

Publish Event

↓

Update Projections

↓

Update Read Models ```

Many moving parts.

---

You now manage:

- Event schemas.
- Event versioning.
- Replay.
- Snapshots.
- Projections.

---

# 24. When is event sourcing genuinely worth using?

Good candidates:

---

## Banking

Because history matters.

Example:

Every transaction must be traceable.

---

## Financial trading

Need:

- Complete audit.
- Exact reconstruction.

---

## Insurance

Claims history matters.

---

## Healthcare

Medical record changes require tracking.

---

## Complex business workflows

Where "what happened" is as important as current state.

---

Example:

Loan approval:

```text ApplicationSubmitted

RiskChecked

Approved

Rejected ```

---
```

25. When is it massive overengineering?

Avoid event sourcing for simple systems.

Examples:

Blog application

```text Create Post

Update Post

Delete Post ```

Normal CRUD is enough.

---

## Simple inventory system

```text Product

Quantity

Price ```

No need.

---

## Admin dashboard

Basic CRUD.

---

Problems if overused:

- More code.
- More infrastructure.
- More operational burden.

---

# CQRS + Event Sourcing Together

A common architecture:

```text Command

Client ----------------------→

                    Write Model

                         |
                         |
                         ↓

                  Event Store

                         |
                         |
                         ↓

                      Events

                         |
                         |
              ---------------------

              ↓                    ↓

       Read Model           Other Services

              ↓

           Queries

```

Final Comparison

Traditional Persistence

```text User Action

↓

Update Database Row

↓

Current State ```

---

## Event Sourcing

```text User Action

↓

Create Event

↓

Store Event

↓

Rebuild Current State ```

---

# Core principle to remember:

> Event sourcing treats the history of changes as the source of truth, not the current database state.

A good event-sourced system provides:

```text ✓ Complete history

✓ Auditability

✓ Temporal queries

✓ Easy reconstruction

✓ Powerful debugging ```

But it costs:

```text ✗ More complexity

✗ More infrastructure

✗ Harder maintenance

✗ Requires strong engineering discipline ```

The key question before choosing event sourcing:

> "Do we truly need to know everything that happened, or do we only need to know the current state?"

If the answer is only current state, normal persistence is usually the better choice.